Zeph

Technical guide

Authors: Shane Mahon (22376743),
Sean Sweeney (22408204)

Supervisor: Dr. Geoff Hamilton

Module: CSC1097

Date: 01/05/26

Abstract

Zeph is a proof of concept for a decentralised flight delay compensation platform for the enforcement of EU Regulation 261/2004. This regulation states that EU air passengers are entitled to compensation for delays longer than three hours. In practice, this entitlement is undermined by slow and inconsistent airline claims processes. As a result, many passengers miss out on compensation that they are legally owed.

Zeph addresses this gap by combining a Solidity smart contract deployed on the Polygon mainnet with a Next.js web application, a Supabase PostgreSQL backend, and a custom Node.js oracle worker that bridges our built-in flight landing time API to the blockchain. Passengers register flights on-chain through a MetaMask-signed transaction. The oracle monitors flight arrival times and reports qualifying delays to the contract. Airlines then accept or reject claims within a seven-day window, after which unanswered claims are automatically accepted and compensation is paid out. Rejections require cryptographically hashed evidence, stored off-chain for privacy but verifiable on-chain. Passengers can export a tamper-evident evidence dossier for dispute escalation.

The system demonstrates how a hybrid Web2/Web3 architecture can make a legal entitlement that is broken in practice both transparent and enforceable, serving as a proof of concept for closing a real gap between regulatory rights and their enforcement.

Table of Contents

1. Introduction

This document is the technical guide for Zeph, a decentralised flight delay compensation platform developed as a fourth year project for CSC1097. The design, development, and testing of the application are described in this document. The system's goal is to partly automate and enhance the typically tedious and slow process of requesting compensation for delayed flights under EU Regulation 261/2004. Zeph combines a Solidity smart contract on the Polygon mainnet with a modern web stack (Next.js and Supabase) along with a custom oracle worker, providing a hybrid Web2/Web3 architecture. This setup shows how EU 261 could be enforced fairly and transparently using blockchain technology.

This guide walks through the building of the project. Section 2 details the project motivation and real-world problems Zeph aims to address. Sections 3 and 4 cover research and design work. Section 5 describes the implementation across each layer. Section 6 highlights representative code from different areas within the project and Section 7 discusses the main technical problems encountered during development and how they were resolved. Section 8 covers the testing of the system. Section 9 talks about the results of the project, and section 10 discusses future work.

2. Project Motivation

Zeph was created to address a clear gap between a passenger's legal right to compensation and their ability to actually receive it in practice. Under EU Regulation 261/2004[1] (EU 261), passengers are entitled to compensation when a flight arrives more than three hours late, unless the airline can prove that extraordinary circumstances caused the disruption. Although this right exists in law, the real-world claims process is usually slow, confusing and opaque. Passengers have to determine their own eligibility, gather evidence themselves, contact the airline through inconsistent channels, and then wait and hope for a response. Even when a response does come, the lack of transparency means claims can easily be rejected incorrectly. Because of this poorly designed system, passengers often resort to using private claim-handling companies (AirHelp, Flightright, Skycop, ClaimFlights, etc)[2], [3], [4], [5] when their flight is delayed. These companies usually take 20-30% of the compensation payout. The existence of so many of these claim companies shows just how badly the current system operates.

This creates major inefficiencies and unfairness in the current system. If a law exists to protect passenger rights then the means of fair and proper enforcement of that law should also exist.

Airlines have little incentive to make compensation easy to claim. As a result, many passengers either don’t know they are entitled to compensation or give up because the process is too time consuming or confusing. Even when claims are valid, there is often no simple way for passengers to verify what stage the claim is at, what evidence has been submitted, or whether the airline is acting fairly or in a reasonable timeframe. The problem is therefore about trust, accountability, and accessibility in the enforcement process of this regulation and not in the law itself.

Zeph stands as a proof of concept for using technology to close that enforcement gap. It shows how blockchain technology combined with a web application and off-chain data storage, can make the claims process more visible, tamper-evident and organised for everyone involved. By recording the important claim events in a verifiable way, forcing airlines to respond within a defined window, and automating parts of the delay checking process, Zeph reduces friction for passengers and makes compliance easier to monitor. It's a demonstration of how digital systems can help bridge the gap between legal entitlement and real-world enforcement.

3. Research

The research phase for this project was practical rather than theoretical. The goal of the research was to identify constraints that the implementation would need to mitigate, and find a suitable solution to meet those constraints. Four main areas were investigated: the regulatory rules that would drive the system's business logic (3.1), the suitability of blockchain for different parts of the architecture (3.2), the oracle problem and how to bridge real-world flight data on-chain (3.3), and the supporting Web2 technology stack (3.4).

3.1 Regulatory context and domain rules

The legal basis for the project is EU Regulation 261/2004. The most important outcome of the research was not just learning about the compensation, but identifying the exact rule that would drive the state machine: flights delayed by 180 minutes or more should qualify for compensation. That threshold directly appears in the implementation in two places:

1. In the Solidity contract as DELAY_THRESHOLD_MINUTES = 180 in Zeph.sol.

2. In the server side flight processing logic where the landing and auto-accept routes check whether delay_minutes >= 180.

This gave the system a single business rule around which multiple components could be aligned. The database, the oracle, the airline dashboard, and the smart contract all treat the 180 minute threshold as the event that converts a simple booking registration into an active compensation claim.

The research also highlighted that airlines can reject claims on exceptional-circumstance grounds such as weather disruption or air traffic control restrictions. That influenced the rejection design. Instead of a simple boolean accept/reject button, the platform requires supporting evidence for rejection and stores both human-readable evidence data and cryptographic hash. This mirrors the real world need for an airline to justify why compensation is not owed.

3.2 Blockchain suitability

The second major research area was whether blockchain should be used at all, and if so, for which parts of the system. A naive design would try to put flight bookings, flight schedules, personal data, and claim evidence directly on chain. Research showed that this would be a poor fit for several reasons:

1. Personal data such as passport numbers and email addresses should not be written on a public ledger[6]
2. Query-heavy operational data such as bookings and routes is easier to manage in a relational database.
3. File evidence such as PDF reports is inefficient and expensive to store on chain.
4. Smart contracts cannot fetch real-world flight data by themselves.

This led to the hybrid design. Blockchain was restricted to the steps where immutability and verifyability matter most:

- Passenger registration of a claim against a flight
- Oracle publication of whether the flight delay exceeded the threshold.
- Airline acceptance or rejection of the flight-level claim decision.

Everything else remains off-chain: bookings, user authentication, route lookup, evidence files, and dashboard rendering.

3.3 The Oracle Problem

The most important blockchain specific finding was the oracle problem. Solidity contracts operate deterministically and therefore cannot make HTTP requests or query an external flight API or database during contract execution. This meant the

contract could never “look up” the actual arrival time of a flight on its own. This meant we needed an external component to fetch this data and feed it to the smart contract.

This led us to making the Oracle Worker implemented in oracle-worker.mjs. Rather than adopting a decentralised oracle network such as Chainlink for the prototype, we built a custom worker for the following reasons:

1. The worker needs to read from our own database, not from third party sources.
2. A custom script is faster to develop and easier to reason about.
3. The trust assumption is acceptable in a prototype because the worker is not hidden, its behaviour is explicit and examinable.

The research trade-off was clear. We accepted centralisation at the oracle layer in exchange for development simplicity, while still preserving an on-chain record of the oracle’s output.

3.4 Tech stack

The off-chain stack was chosen after comparing the practical requirements of the project with what modern frameworks provide.

3.4.1 Next.js

Next.js[7] was selected because it allowed us to keep frontend pages and backend API routes in one codebase. This mattered because many actions in this project cross the browser/server boundary. For example, the browser collects a booking form, a Next.js API route validates it and writes to the database, the browser later connects MetaMask and signs a transaction, and another Next.js API route finalises the registration in the database. Using Next.js let us implement these flows without splitting the project into separate backend repositories.

3.4.2 Supabase

Supabase[8] was chosen because it combines PostgreSQL, storage, and authentication in one managed platform. This was particularly useful for this project because claim and booking data needs relational joins and indexed lookups, airline rejection reports need object storage, airline-only and passenger-only routes require authenticated users, and server-side routes need a privileged client (service role key) for controlled writes. Supabase therefore reduced integration overhead while still giving us SQL-level structure and control.

3.4.3 Ethereum and Polygon

We researched both Ethereum and Polygon as our choice of blockchain providers and compared their advantages and disadvantages. Ethereum offered the clearest baseline because it is the most established EVM ecosystem, has strong documentation, and is the default reference point for many Solidity, Hardhat and MetaMask workflows. However, it also exposed practical drawbacks for our project, particularly around transaction cost and the friction involved in relying on temporary development networks and test-token faucets during development.

Polygon was researched as an alternative because it offers the same broad EVM compatibility while reducing transaction cost, making it more practical for a system where multiple user actions must be recorded on chain. The main trade-off is that it introduces an additional network choice beyond the Ethereum path, but in return it makes repeated contract interactions far cheaper and more realistic for a working prototype.

After comparing both options, we selected Polygon. The exact reasons for that final choice, based on how the system behaved during development and deployment and discussed in the implementation section rather than being repeated in full here.

4. Design

This section describes how Zeph was designed before implementation. Zeph was designed as a layered system that would automate EU261 flight compensation claims while keeping the user experience simple for passenger and airline users. The core idea was to split the project into four main layers: client, backend, on-chain, and off-chain. We chose this structure so users could interact with an easy to use interface, while the blockchain logic and background work happened behind the scenes. This separation also meant that each layer could be designed, tested and implemented independently.

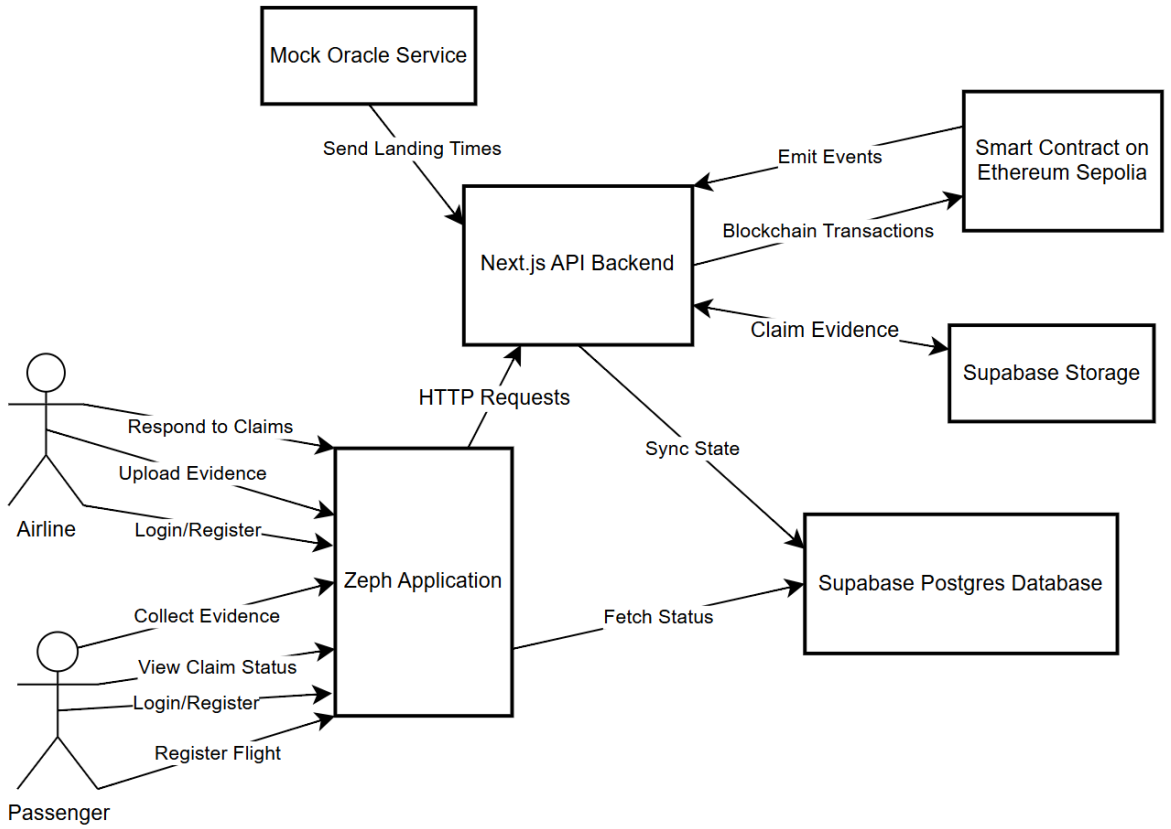

Figure 1 - Context Diagram

The context diagram above shows the high-level relationships between the system's actors and components. The two primary users, passengers and airlines, interact with the Zeph web application through a browser. The application communicates with a backend API, which in turn coordinates between the smart contract, the database, and the oracle service. This diagram showed the boundaries of the system early on and made clear which parts of the process were automated and which required user action.

The client layer was designed to provide two main interfaces. One for passengers and one for airlines with role separation enforced after login. Passengers would be able to register flights, connect a MetaMask wallet, track claim progress, and collect evidence if needed. Airlines would be able to review and respond to claims (within seven days), and provide evidence for rejecting claims. The roles were separated in the design because each user type needed different permissions and views of the system.

The backend layer was planned as the main link between all parts of the system. It would handle authentication, validate requests, receive oracle updates, manage claim data, store evidence files, and send blockchain transactions using ethers.js. This was decided because it allowed the blockchain state to be synchronised back into the database so users could see clear claim updates through the web interface.

The off-chain layer was designed using Supabase Postgres for structured data and Supabase Storage for files. Postgres would store flights, claim statuses, and decision records, while storage would hold files such as airline evidence and generate PDF dossiers. This hybrid approach was chosen as storing everything on the blockchain would be too expensive, inflexible and unsuitable for private passenger data.

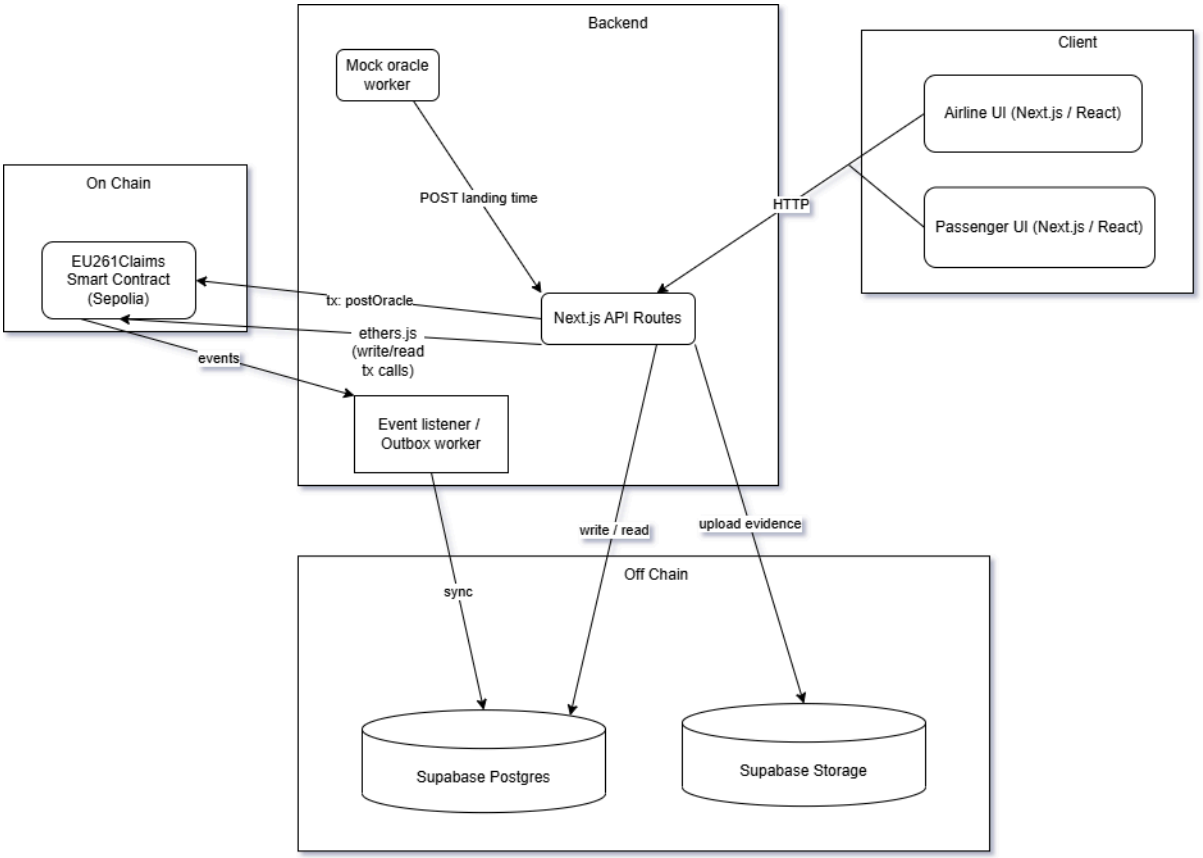

Figure 2 - Component Diagram

The component diagram above shows how these layers connect internally. The oracle worker sits within the backend layer and bridges the off-chain database to the on-chain smart contract by reading newly landed flights from Postgres and submitting delay data as blockchain transactions. The Next.js API routes sit between the client and both the database and the blockchain, ensuring that no client can write directly to either. Supabase Storage is accessed only through the backend, keeping file operations server-controlled

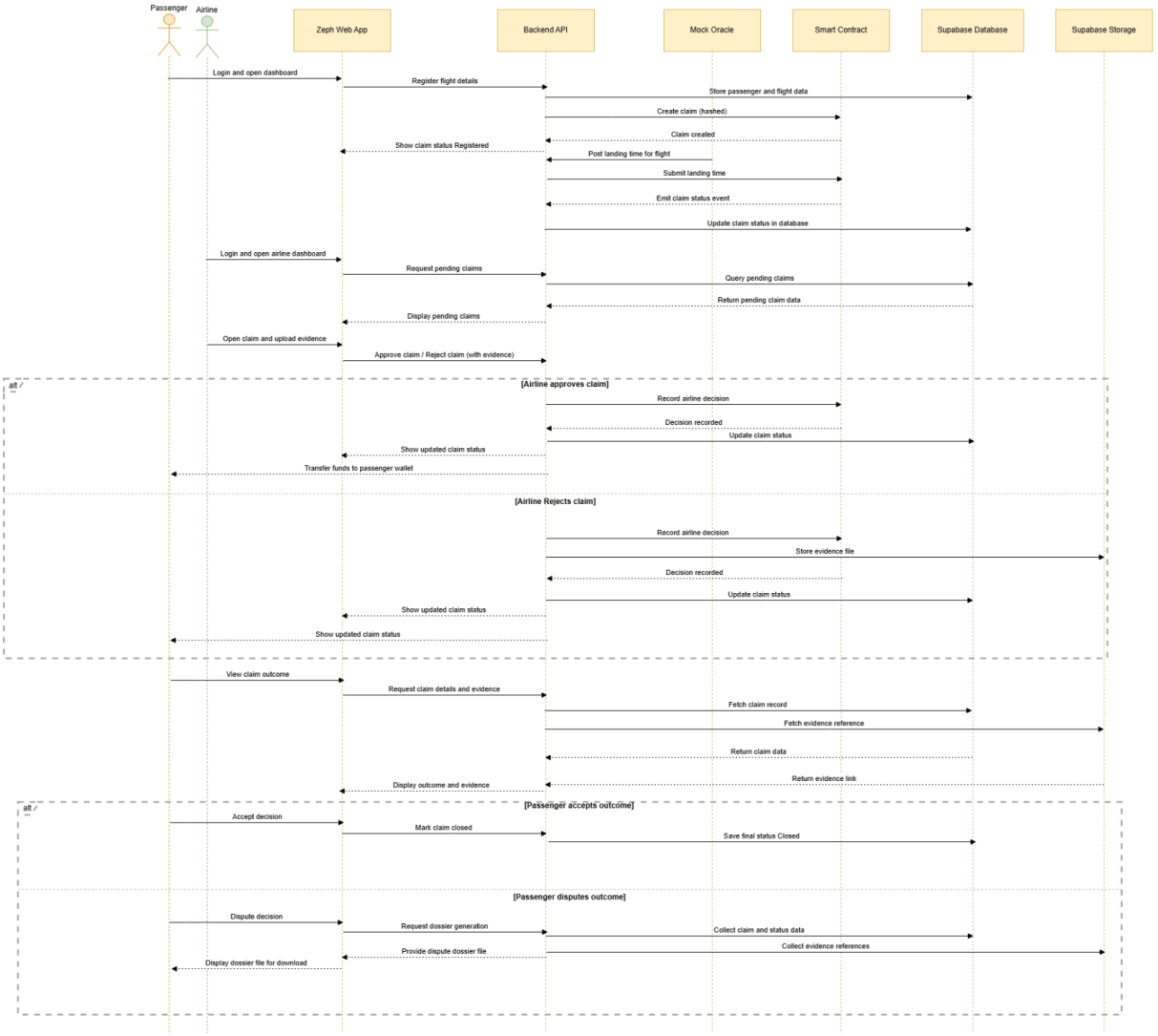

Figure 3 - Sequence Diagram

The sequence diagram above maps the full lifecycle of a compensation claim through the system, from initial login through to dispute resolution. It does not include flight booking but this process can be seen in the user manual.

A passenger logs in and using their booking reference from the flight booking step, registers their flight on-chain via a MetaMask-signed transaction. This is the point at which their claim is formally recorded. The administrator sets the landing time of the flight, this is picked up by the polling oracle service which sends the arrival time to the smart contract. Here, the claim eligibility is decided based on the delay length being greater than or equal to 180 minutes. If it is deemed to be eligible for compensation the claim moves to an awaiting decision state and appears in the airline's claim decision dashboard. The passenger is also notified of the claim status update. The airline then has seven days to either accept or reject the claim with attached rejection evidence of “extraordinary circumstances". If no decision is made within seven days the claim is automatically accepted. Finally if a passenger wishes to dispute a rejection, they can download a full evidence dossier containing all claim data, blockchain

transaction references, and any airline-submitted evidence. This evidence can then be sent as is to the relevant national enforcement body of the jurisdiction.

Security was also a primary concern at the design stage. Three key decisions were made before implementation began. First, role-based access control was designed into the smart contract using distinct roles; AIRLINE_ROLE and ORACLE_ROLE, so that only authorised addresses could report delays or record decisions. Second, reentrancy protection was planned for all contract functions involving state changes, to guard against a class of smart contract vulnerability where a malicious caller re-enters a function before it completes. Third, evidence for rejected claims was designed to be stored off-chain with only a SHA-256 hash recorded on-chain. This protects passenger privacy while still providing a cryptographically verifiable link between the on-chain decision and the off-chain evidence.

Several additional design decisions were made before implementation that shaped the final architecture. Duplicate flight registrations needed to be prevented at both the contract and database level. A mock oracle should be used for flight arrival times rather than a live airline API integration, so that the full workflow could be demonstrated within the project scope. These decisions were made to keep the system realistic and achievable within the time available, while still demonstrating how blockchain technology could improve transparency and fairness in the flight compensation process.

5. Implementation

This section describes the implementation of Zeph. It covers the overall architecture, the key components and how they interact, the flow of data, the different user roles, and the design decisions that were made.

5.1 Overall architecture

A hybrid of Web2/Web3 was used, meaning it combines a traditional web application with blockchain technology. It's split into four main layers:

1. **Web application (Next.js)** - The frontend that users interact with, and a set of serverless backend API routes that handle business logic.

2. **Database (Supabase/PostgreSQL)** - Store off chain data such as bookings, flight details and claim statuses.

3. **Smart Contract (Solidity)** - An on-chain program that provides transparent, tamper-evident records of flight registrations, delay reports and airline decisions.

4. **Oracle** - a background process that bridges the gap between the database and the blockchain by pushing flight delay data on-chain.

5.2 Key components

5.2.1 Frontend

We used React components and styled for responsive design on desktop devices. The interface is split into two main dashboard views. One for passengers and one for airlines. Each one of these was implemented as its own React page while still reusing the same layouts, buttons, alerts, inputs and modal components. This reduced duplication and made it easier to keep the interface consistent across different parts of the platform.

The role based structure was implemented by separating the system into different page routes for passengers and airlines. For example, airline functionality was grouped under pages such as claims.js, while passenger functionality was placed under pages such as register-flight.js. Shared layout wrappers such as AirlineLayout and PassengerLayout were used to give each role its own navigation structure and page styling. This meant that role separation was not just conceptual, but built directly into the routing and component hierarchy of the application

The airline claims page was implemented as a state-driven React component using hooks such as useState, useEffect, useCallback and useMemo. For example, the page stores claim data, loading state, error messages, currently selected filters, evidence collection download state, and reject modal form values in React state variables. When the page loads, a fetchClaims() function is triggered through useEffect() to call the backend API and retrieve the airline's flights and passenger claims. Once returned, this data is rendered into structured cards and tables.

We also implemented the interface so that user actions immediately update the visible state of the page. For example, when an airline accepts or rejects a claim, the component first updates the internal loading state so the relevant button changes to a processing label, then performs the upload, API/blockchain actions, and finally refreshes the claims list from the backend so the updated claim statuses appear on screen. Similarly, evidence collection buttons track which booking reference is currently being processed, so only the relevant row shows a loading state.

Reusable modal and form patterns were an important part of the implementation. Rather than building a separate custom popup for every workflow, shared

components were used for confirmation and form entry. On the airline side, the rejection workflow was implemented as a modal that stores evidence text, URL input, and selected PDF file in React state, then submits those values through appropriate API routes. This approach kept the UI logic close to the user action while still keeping common presentation components reusable.

Styling was implemented to support clear presentation on desktop layouts, which dashboard sections arranged into panels. Flight cards, tables, and status badges. In the airline dashboard specifically, claims are grouped by flight and then displayed per passenger in tabular form, which was implemented by mapping over the fetched array and then mapping again over the passengers for each flight. Status labels such as accepted, rejected, awaiting decision, and registered were rendered through helper functions that convert raw claim states into consistently styled badges. We did this because it centralized how claim states are displayed and avoided repeating the same logic throughout the page.

5.2.2 Serverless API routes (Next.js '/api/*)

The backend logic was implemented using Next.js API routes. These acted as a server side endpoints that frontend can call using "fetch()", but they execute entirely on the server rather than in the browser, This was an important implementation decision as many parts of the system require privileged actions that should not be exposed to the client, such as writing to the database with the Supabase service role key, validating protected actions, uploading files to the storage or coordinating with blockchain related processes.

Each major operation was separated into its own route so that the backend remained modular and easier to test. For example, POST/api/bookings creates bookings and also creates or reuses the associated flight records, POST/api/flights/landing records a flights actual arrival time and calculates delay minutes, POST/api/registration/prepare and POST/api/registration/complete manage two step flight registration flow, POST/api/airline/claims/decide stores airline decisions in the database, and POST/api/airline/claims/upload-report handles rejection report PDF uploads.

These routes were also where most validation was implemented. Incoming request bodies are parsed and checked server side before any write is performed. For example, booking routes validate required route and passenger fields, registration routes validate booking references and transaction hashes, landing routes validate timestamps, and airline decision routes verify that evidence is present when a claim is

rejected. This means that the backend does not assume that the browser will send correct or complete data.

5.2.3 Database (Supabase)

The database layer was implemented using Supabase which provides a managed PostgreSQL database for storing the main operational data of the platform. The schema was designed around the real claim lifecycle, so that each stage of the process could be represented clearly in the database and linked to the next stage through explicit relationships. The system separates route data, flight instances, passenger bookings, blockchain registrations, and airline decisions into different entities, which made the data easier to manage.

'routes' - Stores the predefined route combinations used by the booking system, including the origin city, destination city, flight code and expected journey duration. This table acts as a starting point for a flight. When a passenger selects a journey, the backend looks up the corresponding route and uses that information to determine the correct flight code and calculate the scheduled arrival time.

'flights' - Represents the actual flight instances. Each row corresponds to a specific date and time identified by a generated flight_id. A flight stores its route information, scheduled departure and arrival times, and later, once the flight has landed, the actual arrival time and calculated delay minutes. It also stores oracle related fields such as oracle_processed_at and oracle_tx_hash, which are used to track whether that flight's delay has already been recorded on-chain. The relationship here is that one route can produce many flight records over time, while each individual flight can later be associated with many passenger bookings.

'bookings' - Stores the passenger level booking records. Each booking is linked to exactly one flight through flight_id, which creates a many to one relationship: many bookings can belong to the same flight. The booking record stores the passenger's email address, passport number, route details, and timing information, as well the booking reference number used for later in the claim process. Importantly, a booking by itself does not mean the passenger has registered a blockchain claim. It only means that the passenger has a valid booking in the system.

'registered_flights' - Tracks all flights recorded on- chain and their claim status. In effect, it extends the booking record with blockchain specific claim metadata such as registration status, claim status, transaction hash, contract address, chain ID, and wallet address. This creates a one-to-one relationship with a booking: each booking can have at most one registration record, and that registration record only exists if the passenger completes the wallet registration process. This allowed the system to

distinguish between ordinary passengers who booked a flight and passengers who actively registered their claim on-chain. The registration flow to be tracked in stages, such as pending, confirmed, awaiting decision, accepted, rejected, or auto-accepted.

‘flight_claim_decisions’ - Records the airline’s decisions alongside evidence hashes and blockchain transaction references. Unlike bookings and registration, which operate at passenger level, this table operates at flight level. That means one flight can have many registered bookings and many registered claims, but only one airline decision record. This was a deliberate design decision because in the prototype the airline’s accept/reject decision applies to the flight as a whole rather than being entered separately for each passenger. The decision row stores the outcome, supporting evidence, evidence hash, and related blockchain references. This produces a one-to-one relationship between a flight and its decision record, while that same decision may affect many rows in registered_flights.

Authentication was also handled through Supabase, which provided the user sign up, login and session management. Supbase Auth manages user identities securely and issues session tokens that could be checked by the frontend and backend. This was important because several parts of the system depend on knowing exactly who the current user is, such as registering passengers to their own claim data or ensuring that airline users can only access flights belonging to their airline.

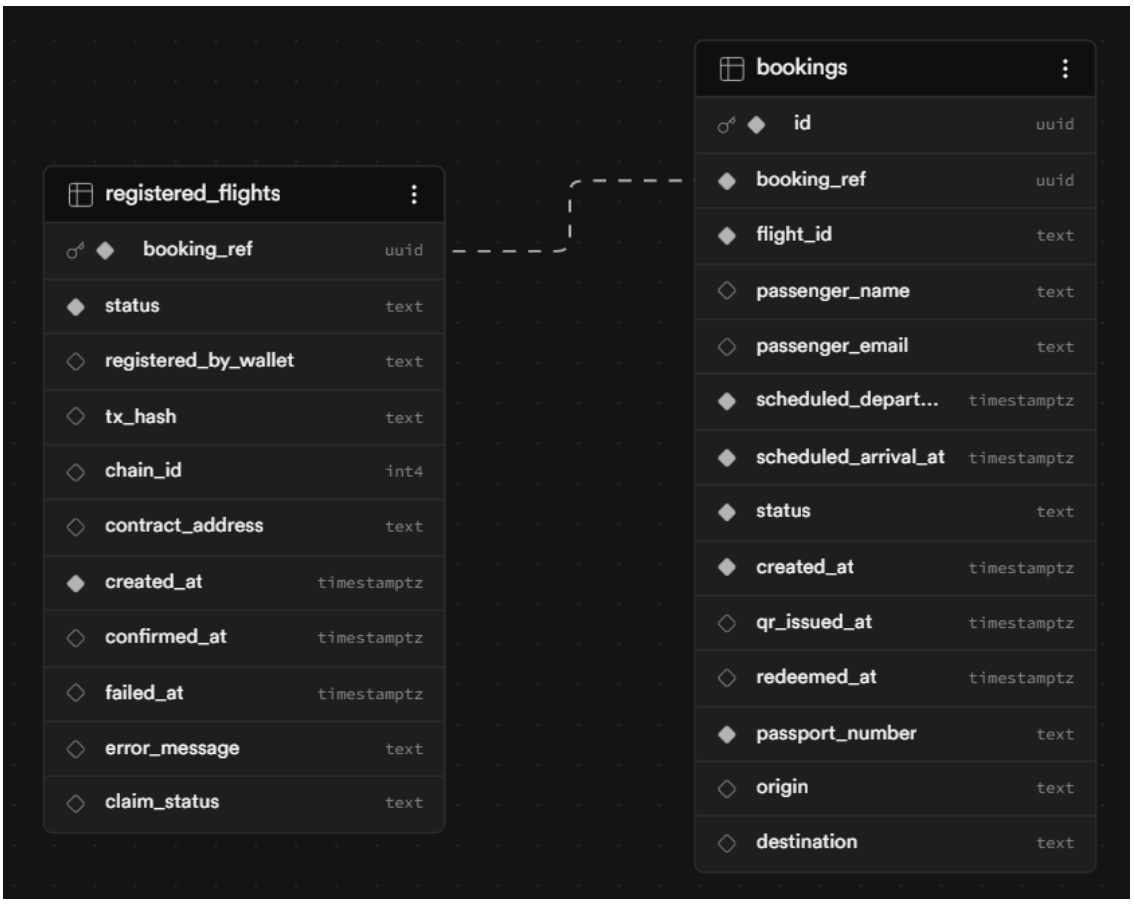

Figure 4 - 'register_flights' and 'bookings' Schemas

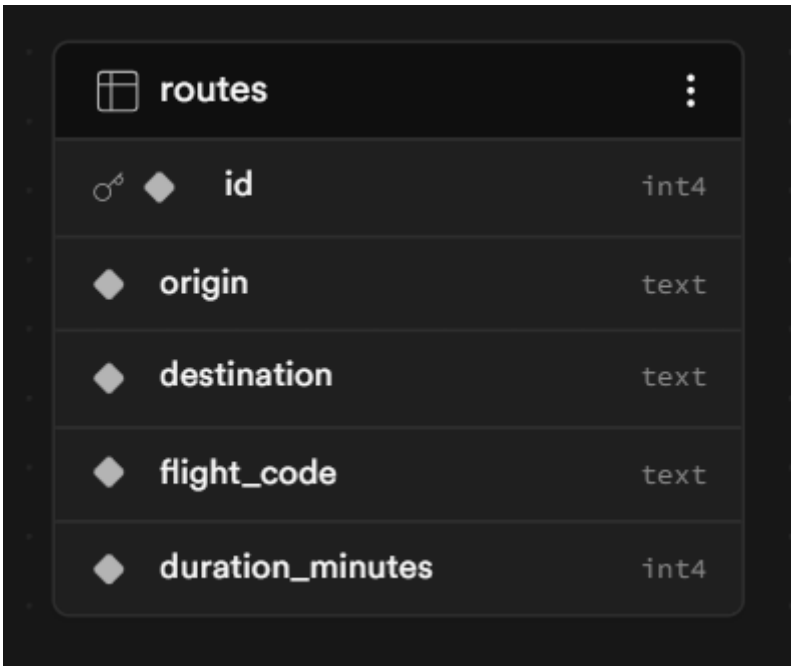

Figure 5 - 'routes' Schema

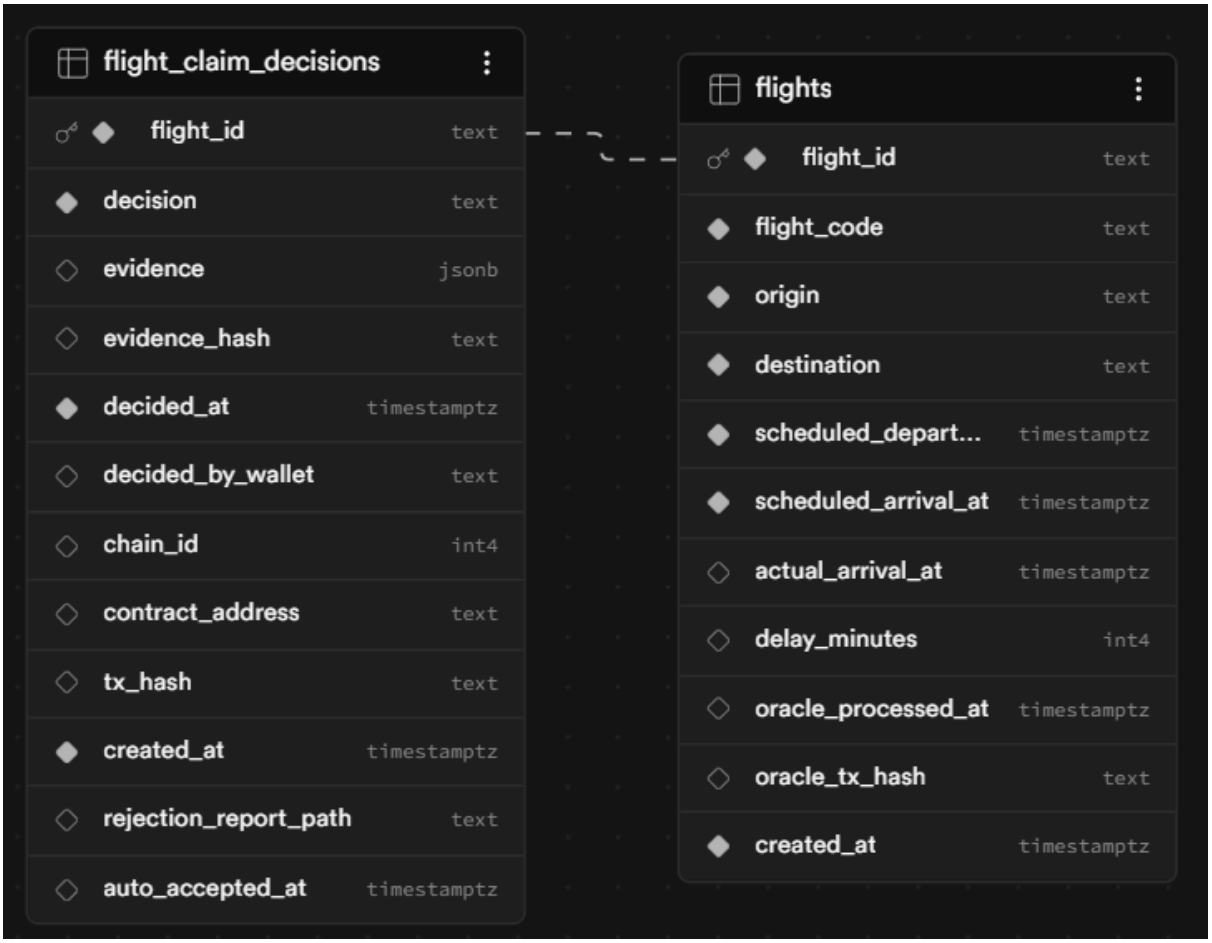

Figure 6 - 'flight_claim_decisions' and 'flights' Schema

5.2.4 Smart Contract

Written in Solidity, the contract was intentionally kept minimal and acts as a state machine rather than a full compensation engine. Instead of storing bookings or passenger data on-chain, it records only the claim milestones that benefit from immutability: passenger registration, oracle confirmed delay status, airline decision and evidence hash.

State is stored in mappings keyed by "keccak256(abi.encode(flightId))". These track whether a passenger wallet registered the flight, whether the oracle marked it as delayed, the airline decision ("None", "Accepted" or "Rejected"), and the evidence hash linked to that decision. This allows the contract to answer the key audit questions without replicating the full off-chain data model.

The contract has 4 main functions:

- **registerFlight()** - A passenger registers their flight with a unique booking reference number
- **oracleReportDelay()** - The oracle reports a flight's delay minutes on-chain

- **airlineDecideFlight()** - The airline accepts or rejects a claim (evidence required for rejection). Its protected by OpenZeplin's nonReentrant modifier guard against re-entrant calls, requires the flight to be already marked as delayed, requires a non-zero evidenceHash if rejecting, and reverts if a decision has already been recorded for that flight.
- **getClaim()** - A read only function to check the status of a claim

5.2.5 Oracle Worker

The Oracle Worker was implemented with a Node.js script that scans the flights table in the database for newly landed flights and submits their delay data to the smart contract using “orcaleReportDelay()”. It can run once or in a continuous watch mode, polling at a configurable interval. This was designed to be independent of the web app so it can run on a separate server or as a scheduled task.

The script connects to Supabase using an admin client and to the blockchain using ethers.js and the oracle wallet. After successfully submitting a delay report, it updates the database with fields such as oracle_processed_at” and “oracle_tx_hash” so the same flight is not processed twice. This gives the system a clear link between the off-chain flight record and the on-chain oracle transaction.

It was implemented as a separate script rather than part of the web app so that it could run independently in the background. The worker supports both a one off mode and a continuous watch mode with a configurable polling interval, making it suitable for manual execution scheduled tasks, or deployment on a separate server.

5.2.6 Evidence Generation Engine

The evidence generation engine was implemented as a server side utility that gathers all claim related information and packages it into a single downloadable ZIP file. We used PDFKit to generate the PDF and JSZip to package it. This package can contain a PDF summary report, a JSON manifest, and if the airline rejected the claim, the uploaded rejection report PDF. The aim was to produce one complete evidence bundle rather than requiring the user to inspect multiple database records and files separately.

The engine pulls data from the database tables, formats it into human readable and structured form, and then packages the result into a ZIP generator. A separate JSON manifest is also created from the same data so that the evidence package includes a structured machine-readable version as well as a human readable PDF. We also used Node’s crypto module to compute SHA-256 hashes for the evidence payload and for any attached rejection report file.

Once all parts are ready, JSZip is used to create a ZIP archive containing the PDF report, the JSON manifest, and where applicable, the the airline rejection report, The ZIP is generated as an in memory buffer and streamed directly back to the browser, rather than writing temporary file to the disk. This made the feature simpler to manage and suited the stateless design of the Next.js API routes.

5.3 Data flow

The system follows a structured lifecycle from booking to compensation. The key stages are:

5.3.1 Booking

A passenger books a flight on our mocked flight booking system. The API looks up the matching route, generates a unique flight ID and booking reference number, and creates records in both ‘flights’ and ‘booking’ tables.

5.3.2 Flight Registration (On-chain)

The passenger connects their MetaMask[9] wallet and registers their booking on the blockchain. The database updates to “pending”, then the blockchain transaction executes, and only after the successful confirmation is the database updated to “confirmed”. This ensures the database and blockchain never fall out of sync. A diagram of this process is shown in the appendix below.

5.3.3 Flight landing

When a flight lands, its actual arrival time is calculated in POST/api/flights/landing, where the backend reads “scheduled_arrival_at” from the flights table, parses the submitted actual_arrival_at, and computes the difference between the actual arrival timestamp and the scheduled arrival timestamp. The delay is then evaluated by the smart contract. If the delay exceeds 180 minutes, the claim for that flight is automatically moved to “awaiting_decision” status so they appear on both the passenger and airline dashboards.

5.3.4 Oracle Processing

The oracle worker picks up newly landed flights, submits their delay data to the smart contract, and updates the database to confirm the on-chain record matches.

5.3.5 Airline decision

The airline reviews the claim, optionally downloads the evidence package, and either accepts or rejects it. Rejections require evidence (written reason or pdf report). The decision is recorded both in the database and on the blockchain.

5.3.6 Auto-Accept Safety net

To prevent the passenger's claim from remaining in "awaiting decision" indefinitely, if the airline does not respond within 7 days, the claim is automatically accepted, protecting the passenger from indefinite delays. This is handled by the server side route POST/api/cron/auto-accept, which is designed to be triggered on a scheduled flight basis. The route scans the database for flights marked as delayed that landed more than 7 days earlier, and updates these to auto_accepted.

The API route uses the Supabase admin client from getSupabaseAdmin() and is secured by an x-cron-secret header matched against ADMIN_SECRET. When triggered by a scheduled job, the route queries the flights table for flights delayed by at least 180 minutes that landed before that cutoff, excluding flights that have a decision made, then retrieves the related bookings and checks the "registered_flights" table for claims that are still awaiting a decision. If such claims exist, it inserts an auto-accepted decision row and bulk-updates the matching registered flights records to auto_accepted.

Rather than actually waiting 7 days to validate this feature, the feature was tested by pre-aging the flight to 7 days, manually preparing the database and then triggering the cron endpoint directly. During test setup, flights can be backdated (e.g. 8 days ago) by calling the flight landing endpoint with an "actualArrivalAt" timestamp set to 8+ days in the past, allowing the cron job to immediately identify them as expired when run. Unit tests in auto-accept.test.js mock the Supbase client to simulate flights with an arrival time of older than 7 days, verifying that the endpoint correctly identifies delayed flights, excludes those with existing decisions, and auto accepts waiting claims in a single run.

5.3.7 Compensation

Once a claim is accepted, it is reflected in the passenger's balance. This is handled off chain rather than being paid directly by the smart contract. This means the user uses the claim status stored in the database to determine whether a passenger is entitled to compensation, and then calculates the total amount owed from the set of accepted claims linked to the passenger.

The balance is derived from the records in “registered_flights” where claims move through statuses such as accepted or rejected. When a claim reaches an accepted state, it becomes eligible to contribute to the passengers displayed balance. The balance route then totals the qualifying claims and applies the project fixed compensation amount of 300 euro. This value is returned to the frontend and displayed in the passenger dashboard.

This approach was chosen because the project focuses primarily on evaluating claim eligibility rather than building a full on-chain payout system. The smart contract provides the tamper-evident record of the important claim events, while the database and API layer handle the calculation and presentation of the compensation amount owed.

5.4 User roles

The system supports three different roles:

Passenger - Can book flights, register flights and view their claim statuses and compensation balance.

Airline - Can view all delayed flights from that airline, review pending claims and download evidence packages, and reject or accept claims.

Admin - Can manually mark flights as delayed through an admin panel

User roles are determined by the email domain - airline domains (e.g. @ryanair.com, @airfrance.com) are mapped to the airline role, all other users are passengers, and administrators are identified with unique credentials.

5.5 Design decisions and rationale

5.5.1 Why Polygon?

We initially chose Ethereum Sepolia[10] as our choice of blockchain provider, running a local Hardhat node for development. This approach had two significant problems. First, the local chain was not persistent, every time the node was shut down, all deployed contracts and recorded transactions were lost, meaning the contract had to be redeployed from scratch each session. This made development and testing slow and repetitive. Second, Sepolia requires test tokens obtained from public faucets, which are rate-limited and unreliable. This created a dependency on an external service just to run basic transactions during development.

These limitations led us to the conclusion that deploying to a real persistent network was the better approach. We chose Polygon mainnet, an Ethereum Layer 2 network, because it offers the same smart contract compatibility as Ethereum while being significantly cheaper to use. Deploying the contract cost approximately one cent in POL gas fees, and each subsequent transaction; flight registrations, oracle delay reports, airline decisions cost only fractions of a cent. This made it practical to demonstrate the full end-to-end flow on a real public blockchain without accumulating meaningful costs, and it meant the contract state persisted between sessions without any manual redeployment.

5.5.2 Why Supabase?

Supabase provides a full PostgreSQL database with built-in Row Level Security, real time data subscriptions, managed authentication, and file storage- all in one platform. This reduced the number of separate services we needed to manage and allowed us to enforce security policies directly at the database level.

5.5.3 Why a two-phase flight registration?

The prepare/complete pattern for registering flights was designed to prevent inconsistencies. If the blockchain transaction failed after the database was already updated, the system would be in a broken state. By only finalising the database record after receiving a confirmed transaction hash, we ensured that the two systems are always synced.

5.5.4 Why a custom oracle instead of Chainlink?

For this prototype, a custom Oracle was simpler to build and deploy, It reads directly from our database, giving us full control over the data pipeline. In a real production system, a decentralised oracle network like Chainlink[11] would be better to remove the single point of failure.

5.6 Deployment pipeline

The deployment process is handled by a Hardhat script which:

1. Deploys the smart contract to the Polygon mainnet
2. Grants AIRLINE_ROLE and ORACLE_ROLE to the specified wallet addresses at deploy time.
3. On local address networks it seeds a test flight with a 240 minute delay for development purposes.
4. Saves a deployment info JSON file and automatically writes the contract address and chain ID into the Next.js environment file so the frontend always picks up the latest deployment.

5. Copies the contract ABI into the frontend directory so the browser can interact with the deployed contract.

5.7 Continuous Integration

The project uses a GitLab[12] CI/CD pipeline that runs automatically on every push and merge request. The pipeline is split into three stages that run in order:

1. **Quality:** Runs a quality check (repo_hygeine) that verifies essential files and folders exist. This acts as a basic safety gate before any heavier jobs run.

2. **Test:** Installs the Next.js web application dependencies and runs the full Jest[13] test suite. Dummy environment variables are injected (Supabase URL, contract address, chain ID) so that the tests can run without a live database or blockchain connection. Next, it installs the Hardhat toolchain and runs all smart contract tests, executing the Chai-based test suite against a temporary in-memory Hardhat node.

3. **Build:** Compiles the solidity smart contract using Hardhat and copies the resulting ABI and bytecode into a 'build/solc/' directory, which is saved as a downloadable CI artifact for 14 days. This job only runs when files relevant to the smart contract have changed, saving CI minutes on frontend only changes.

All jobs run inside 'node:22' Docker containers except repo hygiene which uses a minimal 'alpine' image for speed. The pipeline is triggered on both direct branch pushes and merge request events, meaning every merge request must pass all three stages before it can be merged. This gives us confidence that new code changes will not break existing tests or introduce contract compilation errors.

5.8 Security Considerations

Security concerns were addressed across all layers of the system:

1. **Smart contract security:** The contract uses OpenZeppelin's[14] 'ReentrancyGuard' and 'AccessControl' to prevent re-entrancy attacks and restrict sensitive functions to authorized roles (AIRLINE_ROLE, ORACLE_ROLE).

2. **Evidence Integrity:** Rejection evidence is SHA-256[15] hashed before storage. The hash is also recorded on chain, creating a tamper-evident audit trail.

3. **Database security:** Row level Security restricts data access per user. Backend operations that need elevated access use a server-side service role key that is never exposed to the browser.

4. **API Route Projection:** The cron job is guarded by a secret header. Airline endpoints verify JWT tokens and email domain ownership. Admin routes use a configurable email whitelist.

5. **Polygon gas fee hardening:** The frontend enforces a minimum 25 Gwei priority fee and calculates ‘mavFeePerGas’ dynamically, preventing “gas price below minimum" transaction failures.

6. **Wallet balance pre-check:** Before sending any transaction, the frontend verifies the wallet has funds and shows a clear message if not, rather than letting MetaMask fail with a cryptic error message.

6. Sample Code

6.1 Smart contract delay and decision logic

[solidity]

```JavaScript
function oracleReportDelay(string calldata flightId, uint256 delayMinutes)
    external
    onlyRole(ORACLE_ROLE)
{
    bytes32 key = keccak256(abi.encode(flightId));
    bool delayed = delayMinutes >= DELAY_THRESHOLD_MINUTES;
    flightDelayed[key] = delayed;
    emit OracleDelayReported(flightId, delayMinutes, delayed);
}

// from airlineDecideFlight()
require(flightDelayed[key], "Flight not delayed");
require(flightDecision[key] == FlightDecision.None, "Decision already
recorded");

if (!accept) {
    require(evidenceHash != bytes32(0), "Evidence required");
}
```

This snippet shows the core business rules of the system. The oracleReportDelay function decides whether a flight qualifies as delayed by comparing the reported delay against the contract threshold. Then later on when the airline responds, the contract checks if the flight was actually delayed, prevents duplicate decisions, and forces evidence to be submitted if the airline rejects a claim. This matters because it

includes the most important compensation rules inside the smart contract, making them transparent and difficult to alter unfairly. In the wider system, this is the part that makes the EU261 rule enforceable.

6.2 Oracle worker bridging database and blockchain

JavaScript

```
const { data: pending } = await supabase
  .from('flights')
  .select('flight_id,delay_minutes')
  .not('actual_arrival_at', 'is', null)
  .is('oracle_processed_at', null)
  .order('actual_arrival_at', { ascending: true })
  .limit(batchSize)

for (const flight of pendingList) {
  const tx = await contract.oracleReportDelay(flightId, delayMinutes)
  await supabase
    .from('flights')
    .update({ oracle_processed_at: processedAtIso,
              oracle_tx_hash: tx.hash
             .eq('flight_id', flightId)
})
  await supabase
    .from('registered_flights')
    .update({ claim_status: targetClaimStatus })
    .in('booking_ref', bookingRefs)
}
```

This block is important because it shows how Zeph connects off-chain and on-chain data. The oracle worker checks Supabase for landed flights that haven't been processed on-chain. It then sends the delay information to the smart contract and updates the database so the flight is not reprocessed. After that, it changes the status of related claims in the database so the web application can immediately reflect the result. In the wider system, this is the bridge that keeps the blockchain and the client side in sync.

6.3 Airline evidence hashing and claim decision handling

JavaScript

```
const evidenceJson = evidence && typeof evidence === 'object' ? evidence : null
const evidenceHash = evidenceJson ? sha256Hex(JSON.stringify(evidenceJson)) :
null
```

```
const { error: upsertError } = await supabase
  .from('flight_claim_decisions')
  .upsert({
    flight_id: flightId,
    decision,
    evidence: evidenceJson,
    evidence_hash: evidenceHash,
    rejection_report_path: decision === 'rejected' && rejectionReportPath ?
    rejectionReportPath : null,
    tx_hash: txHash || null,
  }, { onConflict: 'flight_id'
})

const newClaimStatus = decision === 'accepted' ? 'accepted' : 'rejected'
await supabase
  .from('registered_flights')
  .update({ claim_status: newClaimStatus
  .in('booking_ref', bookingRefs)
})
```

This snippet is important because it shows how Zeph handles an airline rejecting or accepting a claim. Instead of storing a larger evidence file directly on-chain, the system stores the full evidence off-chain and generates a hash of it. The same code applies the final decision to all registered passengers on the flight. This matters because it demonstrates the wider design of the project where blockchain is used for trust and verification, while Supabase is used for practical storage and workflow management.

6.4 Two-Phase prepare/complete registration pattern

JavaScript

```
// Phase 1 — prepare (server validates booking and reserves a pending row)
  const prepRes = await fetch('/api/registration/prepare', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ bookingRef }),
  })
  const { flightId } = await prepRes.json()

  // Phase 2a — on-chain transaction (client-side, MetaMask signs and
broadcasts)
  const result = await registerFlightOnChain(flightId)

  // Phase 2b — complete (server confirms the row with the transaction hash)
```

```
await fetch('/api/registration/complete', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    bookingRef,
    txHash: result.transactionHash,
    chainId,
    contractAddress: process.env.NEXT_PUBLIC_CONTRACT_ADDRESS,
  }),
})

// prepare.js — upserts a pending row to reserve the booking
await supabase
  .from('registered_flights')
  .upsert(
    { booking_ref: bookingRef, status: 'pending', tx_hash: null },
    { onConflict: 'booking_ref' }
  )

// complete.js — upgrades the row to confirmed once the tx lands
await supabase
  .from('registered_flights')
  .upsert(
    { booking_ref: bookingRef,
      status: 'confirmed',
      tx_hash: txHash,
      confirmed_at: now,
      registered_by_wallet: registeredByWallet
    },
    { onConflict: 'booking_ref' }
  )

await supabase
  .from('bookings')
  .update({ redeemed_at: now })
  .eq('booking_ref', bookingRef)
```

This snippet is important because it shows how Zeph safely bridges an off-chain booking reference to an on-chain transaction without risking duplicate registrations. The prepare step validates that the booking reference exists and has not already been confirmed, then writes a pending row to the database to reserve it. The blockchain transaction then happens entirely on the client side, where the passenger's wallet signs and broadcasts it. Once that succeeds, the complete step upgrades the row to ‘confirmed’ and attaches the transaction hash and wallet address, then marks the booking as ‘redeemed’ so it cannot be used again. This

matters because the blockchain transaction cannot be rolled back if something goes wrong afterwards, so the two-phase structure ensures the database always reflects reality. Either the registration is still in progress, or it is fully confirmed with a verified on-chain reference. A diagram showing this process is shown in the appendix.

7. Problems Solved

We had to overcome several technical challenges during development:

7.1 State synchronization between Web2 and Web3

We had to ensure that the database directly reflected the state of the smart contract, which was a challenge. We solved this by designing a two step, idempotent workflow rather than single write. First, api/registration/prepare checks that the booking exists and upserts a registered_flights row with a status='pending'. Only after this does the frontend call registerFlight() through MetaMask. Once the transaction succeeds and a real transaction hash is available, POST/api/registration/complete validates the booking reference and transaction hash, then upserts the same row to status = 'confirmed' and stores the chain metadata.

This does not provide rollback in the strict distributed sense. If the chain transaction succeeds but later database write fails, the on-chain operation is idempotent and recoverable: the successful transaction hash can be resubmitted to the "complete" endpoint, which uses upsert(..., {onConflict: 'booking_ref'}) to safely apply the same final state again without duplicating the registration.

7.2 Handling inactive airlines

We noticed early on that if an airline ignored a valid claim, the passengers request would stay pending indefinitely. To address this we implemented a cron job that automatically accepts a claim if it is older than 7 days, resolving the bottleneck between the passenger and the airline, as we discussed in the implementation section in more detail.

7.3 Evidence generation and file handling

Airlines need structured proof to review claims, including flight data, blockchain transaction hashes and passenger details. We built a tool that dynamically creates a PDF summary and packages it into a ZIP file alongside a JSON manifest with SHA-256 integrity checksums. Additionally, handling file uploads in a serverless environment was tricky as Next.js routes don't natively support multipart uploads well. We wrote a

custom raw buffer parser to extract uploaded files directly from the HTTP request body without relying on external libraries.

7.4 Blockchain transaction UI

A major usability issue during development was that blockchain transactions do not behave like normal web requests. such as flight registration depend on MetaMask prompts, network confirmation, gas availability, and block mining time, all of which can fail in ways that are not automatically visible. Early versions of the system gave poor feedback when a user rejected a MetaMask prompt, had insufficient funds for gas, selected a wrong network, or was left waiting while a transaction remained pending.

We mitigated this by adding explicit error and loading state handling around the blockchain calls in the frontend, particularly in the contract utility layer. Calls such as registerFlight() were wrapped in try/catch blocks so that the specific failure cases could be converted into UI pop-up messages. React state was used to reflect pending transaction status, such as airline claims dashboard storing values like deciding, loading, and error in useState, and updating them before and after the blockchain calls. When a transaction is in progress, buttons are disabled and their labels change to “Processing...” or similar text, so that the user can see that the transaction is still pending.

7.5 Oracle Reliability

We had to ensure that the oracle correctly processed flights without missing any or processing the same flight twice. This required careful database flagging, marking each flight with ‘orcale_processed_at’ timestamp, and the oracle only querying for flights where this field is null. After successfully submitting and confirming the on-chain transaction, it marks the flight as processed.

7.6 Polygon gas fee rejections

During testing on Polygon mainnet, transactions were frequently rejected with “gas fee price below minimum” errors because MetaMask’s default priority fee was too low for Polygon’s fee requirements. We solved this by hardcoding a minimum priority fee of 25 Gwei and dynamically calculating ‘maxFeePerGas’ from the current base fee. This ensured transactions were consistently accepted without overpaying.

7.7 Airline role auto-grant tradeoff

The smart contract requires a wallet to hold ARILINE_ROLE before it can accept or reject a claim. Initially this meant that an admin had to manually grant the role via a CLI

command after every contract redeployment. We solved this by adding a server-side API route that automatically grants the role when an airline wallet attempts its first decision. This was a deliberate trade-off, it slightly reduces the manual oversight of role assignment in exchange for much smoother airline onboarding experience. The tradeoff is acceptable because the grant is still restricted to wallets authenticated with a verified airline email domain.

8. Testing

8.1 Testing Strategy

Testing Zeph required a different approach to what would normally be used for a standard web application. The system combines a Next.js interface, a Supabase database, a Node.js oracle worker, and a Solidity smart contract. Testing had to cover traditional software concerns and blockchain behaviour. The testing strategy was split between multiple layers to cover all of the project's components. The subsections below cover each testing type in turn: user testing (8.2), unit testing of isolated pure functions (8.3), API-level testing (8.4), smart contract testing (8.5), integration testing (8.6), UI testing (8.7), end-to-end scenario testing (8.8), and error handling and edge cases (8.9).

The two main testing frameworks used were Jest and Hardhat with Chai. We used Jest for the web application testing because of its support for JavaScript and Node.js, mocking system, and because it can test API routes without needing a running server. Hardhat with Chai was used for the smart contract testing because it provided a local blockchain environment. This meant that we could deploy and interact with the contract during tests without using real funds or connecting to an external network.

We tested each layer first and then validated the connections between them. API routes were tested individually to confirm that they returned correct responses, handled invalid inputs properly, and interacted with the database as we expected. The smart contract was tested to confirm that its rules around delay thresholds, role-based access, and decision logic were enforced correctly. This separation between them made it easier to see failures and it also reduced the risk of one layer's problems masking another's.

The project is a proof of concept rather than a production-ready platform, so the testing strategy prioritised correctness of the core business logic over performance or load testing. In total, 160 tests were written, covering pure utility functions, the full API layer, and the smart contract.

8.2 User Testing

User testing was conducted to validate the usability and intuitiveness of the application across all three user roles: Passenger, Airline, and Admin. The goal was to identify friction points in the UI and gather feedback to help with final improvements before submission.

Testing was carried out in accordance with DCU Faculty Research Ethics Committee (F-REC) guidelines. All participants provided informed consent prior to testing. No personally identifiable information was collected during the session or survey. In total our study had 10 participants.

8.2.1 Methodology

Each participant was given a printed participant information sheet and signed an informed consent before the session began. They were then provided with the zeph user testing guide, which outlined four sequential tasks covering each user role:

- **Task 1 (Passenger):** Book a flight, create a Zeph account, register the flight for compensation.
- **Task 2 (Admin):** Log in to the admin dashboard and set the flight's actual arrival time to simulate a delay of over 180 minutes.
- **Task 3 (Airline):** Log in to the airline dashboard, accept or reject the compensation claim, and explore the airline views.
- **Task 4 (Passenger):** Return to the passenger account, attempt to download the evidence report, and explore the passenger dashboard.

Login credentials for airline and admin roles were provided and the participants were asked not to use any real personal information when creating a passenger account. After completing the tasks, participants filled in an anonymous survey via Google Forms.

The survey used 1-5 Likert scale questions grouped by task and role, followed by two open-ended questions for qualitative feedback. The full list of the questions can be found in the appendix.

8.2.2 Results

8.2.2.1 Quantitative Results

#	Question	Min	Max	Mean
1	**Register flight (passenger)**	3	5	**3.9**
2	**Create account / Login (Passenger)**	4	5	**4.6**
3	**Record flight delay (Admin)**	3	5	**4.2**
4	**Admin interface**	2	4	**3.4**
5	**Find flights (Airline)**	3	5	**4.5**
6	**Accept / Reject (Airline)**	4	5	**4.6**
7	**Airline profile**	3	5	**3.8**
8	**Access evidence (passenger)**	2	5	**4.1**
9	**Passenger profile**	2	5	**4.1**
10	**Visual design / layout (overall)**	3	5	**4.0**
11	**App kept user informed (overall)**	3	4	**3.7**

Overall average = **4.1**

8.2.2.2 Qualitative Results

Feedback from question 11 and 12:
(*Anything confused you / missing in interface?*), (*Any other suggestions for improvement?*)

1. **Admin interface accessibility.**
   Participants reported issues with the admin set-flight workflow; they noted that the absence of an "Admin" option on the login page forced them to select "Passenger" to log in to an admin account, which felt incorrect. It was also highlighted that the native calendar icon on the actual-arrival input was effectively invisible because it was rendered in black against a dark input background. Both issues blocked first-time admin users from completing their task without trial and error.

   - *"There was no 'admin' button to login to the admin account."*

- *"I couldn't see the calendar button when entering the time of flight arrival as admin because the input box was black."*
- *"The explainer section in the admin account was a bit confusing, could be improved."*

2. **Lack of contextual information and guidance.**
   Multiple participants raised some form of "I don't know what to do next" feedback, but in different parts of the application. One participant did not understand the purpose of the evidence report once it had been downloaded. A second pointed out that there was no in-app explanation of how Zeph's flow related to having booked a flight elsewhere. Two more reported that several pages lacked supporting detail, particularly once logged in, and felt the application assumed prior knowledge of the EU passenger-rights process.

   - *"I don't know what you would do with the evidence report."*
   - *"Understanding having to book your flight before logging in. Probably worth having a disclaimer on the landing page or similar to ensure users understand this."*
   - *"Some pages lacked detail or more info."*
   - *"More information about Zeph even when logged in."*

3. **Hidden claim states on the passenger dashboard.**
   One participant reported that they were unable to see the outcome of their claim on the My Claims page, because the page defaulted to a filter that hid claims in terminal states (accepted, rejected, auto-accepted). The participant only located the claim after manually switching to the "Accepted" filter, which is a poor experience for a user simply wanting to check the result of a claim they registered earlier.

   - *I checked my profile to see if my appeal was accepted or declined and "when I checked the section for it, I couldn't see it at all. I had to click specifically into accepted appeals, should have been shown straight away."*

4. **Onboarding and beginner guidance.**
   Multiple participants independently asked for some form of in-app guide explaining the application's overall flow and EU passenger-rights process. Similar to the feedback in point 2 of question 11.

- *"Some general instructions could be useful for training purposes."*
- *"I would recommend a sort of beginner guide, perhaps on sign up, so that users know the exact flow of how to operate the app. Maybe just even a single hardcoded page with instructions."*

5. **Other individual suggestions.**
   The remaining comments did not cluster into a theme, but are worth noting. Two participants suggested making airline analytics more accessible. Specifically the ability to click through summary stats on the airline profile to see the underlying claims. One participant suggested signing users in automatically after account creation rather than requiring a separate login step, and one suggested exploring voice input as an accessibility enhancement. These suggestions are addressed individually in section 8.2.4.

   - *"Make stats easier to see and more accessible."*
   - *"You couldn't click into the stats on the airplane profile, should show details of all the different appeals."*
   - *"One other thing I'd suggest would be automatic sign in after the user registers for the first time."*
   - *"I liked the UI; maybe a voice input feature could make it easier."*

8.2.3 Analysis

The Likert results indicate that participants generally found Zeph intuitive to navigate and use, with the highest scores clustering around airline features and account creation/login. This is somewhat consistent with the qualitative feedback, where no participant raised concerns about the airline functionality, however there was still feedback regarding account creation even though it scored well in the quantitative questions.

The lower scores were concentrated on the admin-facing tasks and the lack of clear information directing users as certain stages. The qualitative responses explain these dips clearly. For Q3 and Q4, two issues compounded: the native calendar icon on the datetime-local input was rendered in black against a dark input background, making it effectively invisible, and the wording of the admin's information banner included regulatory detail that participants reported was confusing rather than helpful.

A separate friction point emerged around the login flow. Participants logging in as the admin role were required to select the "Passenger" radio button, since no admin option existed. While this did not break the flow (the redirect logic correctly identified

admins by email), participants found it counter-intuitive and one explicitly flagged it as missing.

The feedback on the My Claims page revealed a default-state issue rather than a discoverability one. The page initially loaded with a filter that hid claims in terminal states (accepted, rejected, auto-accepted), which meant participants checking the outcome of a claim saw an empty list and had to manually switch filters before the relevant claim appeared.

Finally, two participants suggested some form of beginner guide or onboarding instructions, and one specifically asked for clearer guidance on what to do after a claim was rejected. Both pointed to the same gap: the application explained how to register a flight but not the broader passenger-rights workflow it sits within. This is also reflected in the quantitative feedback with a relatively low score for Q11.

8.2.4 Changes Made in Response to Feedback

The following changes were made on the UserTesting-feedback branch and merged before final submission. Each entry maps to the specific feedback that prompted it.

Admin: Simplified information banner. The information banner above the flight table was rewritten to a single instructional sentence: "Click the calendar icon to set the actual arrival time of a flight, then click Mark Landed." *(Prompted by: "The explainer section in the admin account was a bit confusing, could be improved.")*

Admin: Removed redundant nav link. The "Flight Control" link in the admin top bar duplicated the page heading and was removed for visual clarity. There’s no other pages in the admin interface; the nav bar was redundant here.

Login: Removed role selector, auto-detect from email. The "Passenger / Airline" radio buttons were removed from the login page entirely. The user's role is now determined automatically from their email address, with admin emails routed to /admin, airline emails to /airline/claims, and all others to the passenger portal. This resolves the issue of admins having to select "Passenger" to log in. *(Prompted by: "There was no 'admin' button to login to the admin account.")*

Home page: Booking-first messaging. The hero subtitle on the public landing page was rewritten from a generic marketing statement to a clear instruction: "Book your flight, then log in and register it with Zeph using your booking reference." This makes the application's flow explicit before sign-up. *(Prompted by: "Understanding having to book your flight before logging in. Probably worth having a disclaimer on the load-in page or similar.")*

Passenger: My Claims defaults to All. The default filter on the My Claims page was changed from a curated subset (registered + awaiting decision) to "All", so accepted, rejected, and auto-accepted claims are visible immediately on page load. *(Prompted by: "I had to click specifically into accepted appeals; it should have been shown straight away.")*

New: Help & Support page. A dedicated /help page was added, accessible from the passenger navigation bar to logged-in users. The page includes: a four-step guide for escalating a rejected claim, cards linking to the national enforcement body in each of the six covered jurisdictions (Ireland, UK, France, Germany, Spain, Netherlands), each pointing to that authority's official complaint form, and a frequently-asked-questions section covering eligibility thresholds, the 7-day airline response window, the evidence package, and data security. *(Prompted by: "I don't know what you would do with the evidence report"; "More information about Zeph even when logged in"; "Some general instructions could be useful for training purposes"; "I would recommend a sort of beginner guide.")*

Suggestions not actioned. Three suggestions were considered but not implemented before final submission. The request for clickable stats on the airline profile and a redesigned analytics view was deferred as out of scope for the user-testing iteration; the underlying claims data is already accessible through the Claims tab. The voice-input suggestion was noted as a possible accessibility enhancement but was outside the project's scope. The suggestion to automatically sign users in after registration was not implemented because Supabase's email verification flow requires the user to confirm their email before a session can be issued, and bypassing this would weaken the account verification model.

8.3 Unit Testing

We used the Jest framework for unit testing of pure helper functions. These utilities that take inputs and return outputs without touching the database, the network, or the blockchain. Tests require no mocking and run in milliseconds. The functions covered here include role detection (getRoleFromEmail, isAdminEmail, getAirlineCodeFromEmail), input validation (validateCredentials, validateRegistration), claim status helpers (getClaimStatusExplainer, formatClaimDateTime, getAutoAcceptDeadlineFromDelayReport), and the retry wrapper used by frontend fetch calls (fetchWithRetry).

API route handler tests, which form the bulk of our automated suite, are covered separately in Section 8.4 because they require database mocking and exercise the full HTTP layer.

8.3.1 Role Detection

```
JavaScript
test('returns airline for @ryanair.com email', () => {
expect(getRoleFromEmail('ops@ryanair.com')).toBe('airline')
})
test('returns passenger for a regular email', () => {
expect(getRoleFromEmail('john@gmail.com')).toBe('passenger')
})
```

8.3.2 Claim Status Explainer

```
JavaScript
it('returns explainer text for known status values', () => {
expect(getClaimStatusExplainer('registered')).toMatch(/being monitored/i)
expect(getClaimStatusExplainer('awaiting_decision')).toMatch(/7 days/i)
expect(getClaimStatusExplainer('auto_accepted')).toMatch(/automatically/i)
})
```

8.3.3 Input Validation

```
JavaScript
test('returns error when password is too short', () =>
{ const result = validateCredentials('test@test.com', '12345')
expect(result.valid).toBe(false) expect(result.error).toBe('Password must be at
least 6 characters')
})
```

In total, 47 pure function unit tests run across auth.test.js (33 tests), claimUi.test.js (5 tests), and fetchWithRetry.test.js (9 tests). All pass.

8.4 API Testing

The API layer was tested using Jest with the node-mocks-http library. This allowed us to call API routes directly without a running server. We imported each route as a function and invoked them with mock requests and response objects. The Supabase client was mocked for every test using a shared chainable mock builder, so no real

database connections were made during test runs. The approach kept the tests fast and repeatable during development.

The tests cover seven groups of API routes: bookings, registration, passenger, airline, admin, flights, and cron. For each route we tested both the success path and the failure modes — invalid inputs, missing required fields, unauthorised requests, and database errors. The tests check HTTP status codes, response shape, and that the handler short-circuits at the validation boundary before any database query runs.

For example, POST /api/bookings was tested across eleven scenarios. This included missing fields like departureCity, passportNumber, and email, an invalid date format, a non-existent route, and an airline code that did not match the route's flight code. A successful booking scenario confirmed that a valid booking reference and flight ID were returned.

Tests for POST /api/airline/claims/decide confirmed that a rejection without an evidence object was blocked with a 400 response, that a missing flightId was caught, and that an unsupported decision value such as "maybe" was rejected.

The admin route POST /api/admin/set-flight-delayed had extra tests around authentication. Requests without the x-admin-secret header were expected to return a 403, and the delay calculation logic was verified to confirm that the 180-minute threshold was applied correctly at the API level as well as in the smart contract.

Sample API test cases and outcomes:

Route	Input	Expected	Result
POST /api/bookings	Missing departureCity	400	Pass
POST /api/bookings	Valid full booking body	200, returns bookingRef	Pass
POST /api/airline/claims/decide	Valid acceptance with txHash	200, claimStatus: accepted	Pass
POST /api/airline/claims/decide	Rejection without evidence	400, error mentions "evidence"	Pass
POST /api/admin/set-flight-delayed	Missing x-admin-secret header	403	Pass

8.5 Smart Contract and Blockchain Testing

To test blockchain components we used the Hardhat development environment alongside Chai assertions. We simulated deployment, role assumption, and transaction execution. Tests verified that:

- The contract was deployed with the correct roles assigned to the deployer, airline address, and oracle address.
- Only addresses with the designated ORACLE_ROLE could report flight delays.
- Only addresses with the AIRLINE_ROLE could submit claim decisions.
- The 180-minute delay threshold was enforced at the boundary, including the exact-180-minute edge case.
- Passenger registrations were tracked correctly and could not be duplicated from the same wallet.
- Claim decisions could not be overwritten once recorded.
- Events were emitted on registration and oracle reporting so off-chain consumers can subscribe.

Route	Input	Expected	Result
Constructor	Deploy contract	DEFAULT_ADMIN_ROLE granted to deployer	Pass
Constructor	Deploy contract	AIRLINE_ROLE granted to airline address	Pass
Constructor	Deploy contract	ORACLE_ROLE granted to oracle address	Pass
registerFlight	Passenger registers flight	registered=true	Pass
registerFlight	Successful registration	Emits FlightRegistered event	Pass
registerFlight	Passenger registers same flight twice	Reverts with "Already registered"	Pass
oracleReportDelay	Delay of 200 min	flightDelayed set	Pass

		to true	
oracleReportDelay	Delay of exactly 180 min	flightDelayed set to true	Pass
oracleReportDelay	Delay of 100 min	flightDelayed set to false	Pass
oracleReportDelay	Successful report	Emits OracleDelayReported event	Pass
oracleReportDelay	Caller without ORACLE_ROLE	Reverts (AccessControl)	Pass
airlineDecideFlight	Airline accepts a delayed flight	Decision = Accepted; getClaim returns decision 1	Pass
airlineDecideFlight	Airline rejects with a non-zero evidence hash	Decision = Rejected; evidence hash stored on-chain	Pass
airlineDecideFlight	Decision attempted on non-delayed flight	Reverts with "Flight not delayed"	Pass
airlineDecideFlight	Rejection submitted with zero evidence hash	Reverts	Pass
airlineDecideFlight	Duplicate decision attempted	Reverts with "Decision already recorded"	Pass
airlineDecideFlight	Caller without AIRLINE_ROLE	Reverts (AccessControl)	Pass
getClaim	Full flow: register → oracle delay → airline reject	Returns correct composite state across all fields	Pass

The full suite of 18 tests passes. The final getClaim test is the most important: it composes the entire EU261 enforcement flow into a single end-to-end on-chain scenario (Section 8.8).

8.6 Integration Testing

The integration tests aim to verify that multiple parts of the system worked correctly together. In Zeph this meant checking that the backend's coordination across the database tables, business rules, and external components produced the expected outcome and that the sequence of operations was joined up rather than treated as separate steps.

The integration tests live in the same suite as the API tests and use the same mocking infrastructure. The external services (the blockchain and Supabase) are still mocked at the boundary, but the tests exercise the full internal logic of each handler — how it queries multiple tables, applies business rules, and updates records in the correct order. The focus is on how the pieces of a handler's logic connect, not on whether a single isolated query works.

The clearest example is the auto-accept cron job. This handler works across four database tables: it checks flights for delayed flights past the seven-day window, filters out those with an existing row in flight_claim_decisions, queries bookings to find the affected passengers, and finally updates registered_flights to apply an auto_accepted status. Tests confirmed this full sequence ran correctly end-to-end, including the case where a flight is eligible by age but has already been decided manually and so must be skipped.

A second integration example is the two-phase registration flow. POST /api/registration/prepare validates the booking reference against bookings, checks registered_flights for an existing confirmed registration (returning 409 if found), and writes a pending row before the user is sent to MetaMask. After the on-chain transaction confirms, POST /api/registration/complete updates the pending row to ‘confirmed’, stores the transaction hash, and marks the booking as redeemed_at in the bookings table. The tests cover both halves of this flow and confirm that the database remains internally consistent regardless of which step the user reaches.

Similarly, the claim-status transition route was tested to confirm that when a delay was reported, all matching passenger records in bookings and registered_flights were updated together, not just the first row found.

All integration tests passed. The test runner flags any failure automatically and the full suite can be run at any time to confirm the integrations remain functioning.

8.7 User Interface Testing

We approached UI testing from a user-centric perspective, exercising each role's flow manually in a browser during development. Automated UI testing (Cypress, Playwright, etc.) was out of scope for this project. See Section 8.11.1.

For the passenger flow we verified that booking-reference entry, MetaMask wallet connection, on-chain registration, and claim-status viewing all functioned smoothly, including when the browser window was resized between desktop and mobile breakpoints. The compensation balance screen was tested to confirm that accepted claims displayed €300 per qualifying flight and that the running balance summed correctly across multiple accepted claims. The evidence export flow was checked end-to-end: clicking "Download Evidence" generated a ZIP containing a PDF, a JSON manifest, and any attached airline rejection report, and the downloaded archive opened correctly without corruption.

For the airline flow we validated that the dashboard correctly filtered flights by airline code (e.g., a Ryanair operations user only sees FR flights), that the claim-decision modal rendered the affected passenger list, and that the full rejection process — uploading a PDF report, providing reasoning text, and signing the on-chain transaction — completed without error. We also confirmed that the rejection report was uploaded to Supabase Storage and that its URL appeared correctly on the passenger's claim view.

For the admin flow we manually checked that login, flight lookup by flightId, and the "set flight delayed" action propagated correctly: setting an arrival time more than 180 minutes after the scheduled arrival caused the affected claims to transition into awaiting_decision and appear in the relevant airline's dashboard.

UI helper functions used by these screens (role detection, claim status messaging, input validation) are covered by the unit tests in Section 8.3.

8.8 End-to-End Scenario Testing

End-to-end testing of Zeph was carried out manually during development. The passenger flow covered account creation, flight booking, wallet connection, on-chain registration, oracle delay processing, and claim status updates. The airline flow covered logging in, reviewing a delayed claim, submitting a decision with evidence, and confirming the status changes. The admin flow confirmed logging in, searching for a flight by the flight id and setting a flight as delayed functioned correctly.

The one automated end-to-end scenario is the getClaim test in the smart contract suite. This runs a complete on-chain flow: a passenger registers a flight, the oracle reports a 210-minute delay, the airline rejects with an evidence hash, and getClaim is called to verify the final combined state. This directly validates the core EU261 enforcement flow on-chain.

8.9 Error Handling and Edge Cases

To ensure system resilience, diverse edge cases and error states were rigorously tested across both the automated test suite and manual testing during development.

8.9.1 Missing/malformed inputs

The API layer was tested exhaustively for input validation failures. For each route, every required field was identified and tested when omitted, set to null, set to an empty string, or replaced with an invalid value. For example, POST /api/bookings was tested across eleven scenarios covering a missing departure city, a missing passport number, a route that does not exist in the database, and various combinations of invalid fields, each expected to return a 400 with a descriptive error message. POST /api/registration/prepare was tested with an invalid UUID format for the booking reference, an empty string, and a missing body entirely. The route validates UUID format explicitly using a regex before any database query is attempted. POST /api/airline/claims/decide was tested with an unsupported decision value such as "maybe", a missing flightId, and a rejection submitted with no evidence object, each rejected at the validation layer before any Supabase operation ran. Malformed JSON bodies were also tested by passing unparseable strings to routes that expect JSON, confirming a 400 was returned before handler logic executed. The consistent pattern is that validation failures are cheap, they are caught at the boundary before any database query or blockchain interaction is started.

8.9.2 Blockchain failures

Blockchain interactions introduce failure modes that standard web applications do not encounter. Unlike an HTTP request that either succeeds or fails quickly, a MetaMask transaction can be rejected before broadcast, hang during pending confirmation, fail due to insufficient funds, or revert on-chain after it has already been sent. Each of these states needs to be surfaced to the user clearly rather than failing silently or leaving the UI in a broken state.

The following scenarios were tested manually during development. If a user dismisses the MetaMask popup, the provider throws an error with code 4001 (user rejected). This is caught in a try/catch wrapping the wallet call and displayed as a toast notification: "Transaction cancelled". If the connected wallet has insufficient POL to

cover the gas fee, the provider throws before the transaction is broadcast. This is caught and shown as "Insufficient POL for gas fees". The application also checks that the connected wallet is on Polygon mainnet (chain ID 137) before initiating any transaction; if a user is on Ethereum mainnet or a testnet, they are prompted to switch networks first. A loading spinner is shown between the moment the MetaMask popup is confirmed and the moment the transaction is confirmed on-chain, preventing users from assuming the page has frozen during the typical block confirmation window.

Finally, if the contract rejects the transaction after broadcast, for example, if a passenger tries to register an already-registered flight, the revert reason is extracted from the provider error and shown directly in the UI.

8.9.3 Database outages

All Supabase interactions in the API layer use the { data, error } pattern returned by the Supabase client. When error is non-null, the handler returns a 500 response with the error message rather than attempting to continue with a null or undefined data value.

This was tested using the chainable Supabase mock builder by configuring it to return “data: null, error: { message: 'connection failed' }” for specific query calls and verifying the handler returned the correct 500 response without throwing an unhandled exception.

The key concern being verified was that the Node process did not crash and that other requests were still handled normally after a simulated failure. This is important in a serverless environment where an unhandled promise rejection can poison subsequent requests.

Two specific scenarios were given extra attention: a SELECT failing mid-handler (confirming the route returns 500 without attempting the subsequent UPDATE), and a Supabase UPSERT failing after a blockchain transaction had already been sent. In the latter case the on-chain state would exist but the database would be out of sync. This risk is mitigated by the two-phase registration pattern (Section 5.5.3), which ensures the database write only happens after on-chain confirmation, and by the oracle's oracle_processed_at idempotency flag, which prevents a flight from being reprocessed even if a previous run partially completed.

8.9.4 Duplicate actions

The auto-accept cron job and the airline's manual decision endpoint can both attempt to mark the same claim as accepted. This is a genuine concurrency risk: the cron job runs on a schedule and an airline could submit a decision in the same time window. We tested this by simulating both paths executing against the same claim.

Auto-accept is an off-chain operation: the cron job writes only to the flight_claim_decisions and registered_flights tables and does not call the smart contract, so the two paths converge in the database rather than on-chain. Before inserting, the cron filters out any flight that already has a row in flight_claim_decisions, so once an airline decision exists, subsequent cron runs skip that flight entirely. The airline decides endpoint upserts on flight_id with onConflict: 'flight_id', so concurrent writes to the same flight overwrite the row idempotently rather than throwing a constraint error or creating a duplicate. The on-chain require(flightDecision[key] == FlightDecision.None, "Decision already recorded") in airlineDecideFlight is a separate guarantee that prevents a duplicate airline transaction on-chain (e.g. a double-clicked accept), independent of the off-chain auto-accept path.

We also tested the case where two identical requests arrive nearly simultaneously. For example, a user double-clicking the accept button. The database upsert handles this without error, and the smart contract would revert the second blockchain transaction, leaving the system in a consistent state.

8.9.5 Re-registration prevention

A passenger should not be able to register the same flight twice. This is enforced independently at two separate layers so a failure at one cannot be exploited to bypass the other.

At the smart contract level, registerFlight stores a registration record keyed by the hash of the flight ID and the caller's wallet address. The require(!claims[key][msg.sender].registered, "Already registered") check ensures any second call from the same wallet reverts immediately, and this is directly covered by the Hardhat test suite (see Section 8.5).

At the database level, the prepare.js handler checks for an existing row with status = 'confirmed' for the given booking reference before writing the pending reservation. If one is found, it returns 409 Conflict before the user ever reaches the MetaMask prompt. Additionally, once complete.js runs successfully, it sets redeemed_at on the corresponding row in the bookings table, marking the booking reference as spent. Even if the registered_flights row was somehow removed, a fresh prepare request would be rejected because the booking reference is already marked as redeemed.

The two-layer defence means the database blocks re-registration attempts before any gas is spent, and the smart contract enforces the same rule independently on-chain.

8.10 Test Results

All automated tests pass. The full Jest suite of 142 tests runs without failures along with all 18 Hardhat smart contract tests which also pass. The table below summarises the results by area:

Area	**# Tests**	**Result**
Smart contract	18	All pass
Auth utils	33	All pass
Bookings	11	All pass
Registration	20	All pass
Passenger	13	All pass
Airline	15	All pass
Admin	10	All pass
Flights	19	All pass
Cron	7	All pass
UI utilities	14	All pass
Total	**160**	**All pass**

Several routes and components are deliberately out of scope for the automated suite. The evidence-export route (/api/passenger/claims/evidence) generates PDFs and ZIPs at runtime and would require mocking PDFKit, JSZip, and Supabase Storage to test meaningfully; we covered it through manual end-to-end testing instead. The thin ensure-delay and grant-role airline routes are wrappers around ethers.js calls whose underlying contract logic is already covered by the Hardhat suite. The oracle worker (oracle-worker.mjs) is a long-running standalone process and was tested through manual execution and log inspection rather than unit tests. Browser-side helpers in utils/contract.js depend on a MetaMask provider and were exercised only through manual UI testing.

8.11 Future Testing Improvements + Testing Limitations

8.11.1 Limitations

Due to time constraints, the project relied on manual UI testing rather than automated browser tests, so cross-browser regressions can only be caught by re-running the manual flows before each milestone. Simulating real-world blockchain network congestion, fluctuating gas fees, and block-mining delays is difficult on a local Hardhat node, so some Web3 UX edge cases (long pending-confirmation windows, dropped transactions, fee spikes) could not be reproduced exactly as they would appear on Polygon mainnet.

Specific components were not covered by the automated suite for the reasons listed in Section 8.10: the evidence-export route, the ensure-delay and grant-role thin wrappers, the React page components, the oracle worker, and the browser-side utils/contract.js. Each of these is a candidate for future automation as discussed in Section 8.11.2.

8.11.2 Future improvements

In later iterations, testing could be improved by integrating End-to-End testing frameworks like Cypress[16] or Playwright[17]. Using specialized Web3 testing tools like Synpress[18] would allow us to automate MetaMask wallet interactions and test the complete visual flow from React click to blockchain transactions in a CI/CD pipeline. Integration tests for the oracle worker could be added to verify the full pipeline from database queries through to on-chain transactions and database updates.

9. Results

The finished system successfully demonstrates the core idea of an automated EU261 flight compensation platform. The final prototype delivers a complete working flow across all three user roles. A passenger can create an account, book a flight within the system, connect a MetaMask wallet, register that flight on-chain, track their claim status in real time, receive compensation when a claim is accepted, and export a tamper-evident evidence dossier if they wish to escalate a dispute. On the airline side, the dashboard shows all delayed flights for that airline filtered by airline code, where delayed flight claims can be reviewed and either accepted or rejected with supporting evidence, these decisions are recorded both in the database and on the blockchain. The admin panel allows flight arrival times to be set manually to simulate delays and trigger the compensation workflow.

The smart contract is deployed on the Polygon mainnet. Deploying the contract cost approximately one cent in POL gas fees. All subsequent transactions; flight registrations, oracle delay reports, and airline decisions each cost fractions of a cent.

All 160 automated tests pass across both the Jest and Hardhat suites, as is shown in the table in section **8.10**.

The core EU261 enforcement logic which is the 180-minute threshold along with the seven-day airline response window enforced by the auto-accept safety net, and evidence hashing all function correctly and enforced by the smart contract in a way that neither the airline nor any other party can alter unilaterally. The combination of on-chain enforcement and off-chain practicality demonstrates that the hybrid Web2/Web3 architecture is viable for this problem domain. As a proof of concept, the system deliberately scopes certain features as simulated. Flights are created when a user books a flight unlike in reality where passengers book a seat on an existing flight, flight arrival times are set through the admin panel rather than a live API, and compensation is tracked as a credited balance rather than an on-chain stablecoin transfer or traditional bank transfer compensation. This allows the full end-to-end compensation workflow to be demonstrated cleanly within the project scope. These areas represent natural next steps for a production system and are discussed further in Section 10.

10. Future Work

Although the system successfully does what was intended, future versions could be expanded in several ways:

10.1 Decentralised Oracles

Transitioning from a single, centralised custom oracle worker to a decentralised oracle network (DON) like Chainlink[11] would remove the single point of failure when fetching flight delay data. In the current system, the oracle-worker.mjs script reads flight arrival times directly from our Supabase database and submits them on-chain from a single wallet holding ORACLE_ROLE. A production version using Chainlink Functions would replace this with a consumer contract that calls a real third-party flight status API (such as AviationStack[19] or FlightAware[20]), with the DON coordinator nodes independently verifying the response before it reaches the smart contract. The oracleReportDelay function would need to be updated so that only the Chainlink coordinator address, rather than a single trusted wallet, can call it. This was not done for this project because Chainlink Functions requires LINK token funding, a deployed consumer contract, and a live flight API subscription. None of this is feasible

because flight data is internally controlled through the admin panel for demonstration purposes.

10.2 Zero Knowledge Proofs

Integrating ZKP technology to verify passenger details on-chain without exposing personally identifiable information (PII) to the public ledger would improve privacy. Currently, a passenger's email address and passport number are stored off-chain in Supabase, but they are still directly linked to an on-chain flight registration that is publicly visible. ZKP would allow a passenger to prove that they hold a valid confirmed booking on a specific flight without the contract or any blockchain observer ever seeing their name or passport number. ZK-SNARK[21] frameworks such as Circom/snarkjs[22] or RISC Zero[23] could generate a proof off-chain that the registration transaction would then verify on-chain. This was out of scope for this proof-of-concept as ZKP tooling carries a steep learning curve and introducing proof generation and on-chain verification would substantially increase the complexity of the registration flow.

10.3 Real compensation payouts

Upgrading the system to deliver real compensation payouts, either via stablecoins such as USDC[24] or USDT[25] for a fully on-chain flow, or via traditional bank transfers for passengers who prefer fiat currency. This would make the compensation workflow complete. Currently, compensation is tracked only as a credited balance in the UI and no funds actually move.

The stablecoin path would require airlines to deposit USDC or USDT into the contract as escrow before claims can be processed, with the contract holding those funds and releasing them automatically when a claim is accepted. RegisterFlight would need to accept or verify a corresponding token transfer, and airlineDecideFlight would trigger an ERC-20 transfer[26] call to the passenger's wallet upon acceptance.

The bank transfer path would not require smart contract changes. Instead, the accepted claim_status written to Supabase would trigger an outbound webhook or background job that calls a payment processor, such as Stripe Payouts[27] or a SEPA credit transfer API[28] to send the compensation directly to the passenger's registered bank account. This path is closer to how airlines would integrate compensation in practice and removes the requirement for passengers to hold a crypto wallet.

The two approaches are complementary: the on-chain stablecoin path offers full transparency with no intermediary, while the bank transfer path is more accessible to non-crypto users. Neither was implemented here because stablecoin escrow requires

pre-funded airline wallets and a higher smart contract security bar, while bank transfers require a payment processor account, KYC compliance, and real banking credentials, neither of which is practical for a demonstration system.

10.4 Expanding claim scenarios

Extending the operational logic beyond 180-minute delays to cover outright flight cancellations, lost baggage claims, and tiered compensation based on flight distance would make Zeph a much closer model of the full EU261 regulation. Currently the contract applies a flat compensation amount and only handles arrival delays of 180 minutes or more. EU261 also covers cancellations with less than 14 days' notice and uses a distance-based compensation scale: €250 for routes under 1,500 km, €400 for routes between 1,500 and 3,500 km, and €600 for longer routes. Implementing this in full would require a flightDistanceKm field to be passed at registration, a cancellation flag alongside the delay flag in oracleReportDelay, and tiered payout logic inside airlineDecideFlight. On the frontend, cancelled flights would also need a separate registration path since there is no arrival time to wait for, and the claims dashboard would need to display the applicable distance tier and expected compensation amount per claim.

11. References

[1] *Regulation (EC) No 261/2004 of the European Parliament and of the Council of 11 February 2004 establishing common rules on compensation and assistance to passengers in the event of denied boarding and of cancellation or long delay of flights, and repealing Regulation (EEC) No 295/91 (Text with EEA relevance) - Commission Statement*, vol. 046. 2004. Accessed: Apr. 20, 2026. [Online]. Available: http://data.europa.eu/eli/reg/2004/261/oj

[2] “AirHelp - #1 Air Passenger Rights Experts,” AirHelp. Accessed: Apr. 12, 2026. [Online]. Available: https://www.airhelp.co.uk/

[3] “Skycop - Flight Delay Compensation - Claim up to €600.” Accessed: Apr. 12, 2026. [Online]. Available: https://www.skycop.com/

[4] “ClaimFlights – Your Passenger Advocate for EU261 Claims,” ClaimFlights. Accessed: Apr. 12, 2026. [Online]. Available: https://claimflights.com/

[5] “Claim your Right for Flight Delay Compensation - Flightright.” Accessed: Apr. 12, 2026. [Online]. Available: https://www.flightright.com/

[6] “Art. 5 GDPR – Principles relating to processing of personal data,” General Data Protection Regulation (GDPR). Accessed: Apr. 20, 2026. [Online]. Available: https://gdpr-info.eu/art-5-gdpr/

[7] “Next.js Docs | Next.js.” Accessed: Apr. 20, 2026. [Online]. Available: https://nextjs.org/docs

[8] Supabase, “Supabase Docs.” Accessed: Apr. 20, 2026. [Online]. Available: https://supabase.com/docs

[9] MetaMask, “Home | MetaMask developer documentation,” MetaMask. Accessed: Apr. 12, 2026. [Online]. Available: https://docs.metamask.io/
[10] “Networks,” ethereum.org. Accessed: Apr. 12, 2026. [Online]. Available: https://ethereum.org/developers/docs/networks/
[11] C. Labs, “Chainlink Documentation,” Chainlink Documentation. Accessed: Apr. 12, 2026. [Online]. Available: https://docs.chain.link/
[12] “Get started with GitLab CI/CD | GitLab Docs.” Accessed: Apr. 20, 2026. [Online]. Available: https://docs.gitlab.com/ci/
[13] “Getting Started · Jest.” Accessed: Apr. 20, 2026. [Online]. Available: https://jestjs.io/docs/getting-started
[14] “OpenZeppelin Docs,” OpenZeppelin Docs. Accessed: Apr. 12, 2026. [Online]. Available: https://docs.openzeppelin.com
[15] Madan, “A Deep Dive into SHA-256: Working Principles and Applications,” Medium. Accessed: Apr. 12, 2026. [Online]. Available: https://medium.com/@madan_nv/a-deep-dive-into-sha-256-working-principles-and-applications-a38cccc390d4
[16] “Cypress testing solutions | Cypress Documentation | Cypress Documentation.” Accessed: Apr. 14, 2026. [Online]. Available: https://docs.cypress.io/app/get-started/why-cypress
[17] “Installation | Playwright.” Accessed: Apr. 14, 2026. [Online]. Available: https://playwright.dev/docs/intro
[18] “Why Synpress,” Synpress Docs. Accessed: Apr. 14, 2026. [Online]. Available: https://docs.synpress.io/docs/introduction
[19] “Aviationstack: Real-Time Flight Tracker API - Free Flight APIs.” Accessed: Apr. 14, 2026. [Online]. Available: https://aviationstack.com/
[20] “Firehose (Live Flight Data API) Documentation,” FlightAware. Accessed: Apr. 14, 2026. [Online]. Available: http://www.flightaware.com/commercial/firehose/documentation
[21] “Non-interactive zero-knowledge proof,” *Wikipedia*. Mar. 15, 2026. Accessed: Apr. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Non-interactive_zero-knowledge_proof&oldid=1343664189
[22] “Circom / snarkjs - Iden3 Documentation.” Accessed: Apr. 14, 2026. [Online]. Available: https://docs.iden3.io/circom-snarkjs/
[23] “RISC Zero,” RISC Zero. Accessed: Apr. 14, 2026. [Online]. Available: https://risczero.com
[24] “USDC (cryptocurrency),” *Wikipedia*. Apr. 06, 2026. Accessed: Apr. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=USDC_(cryptocurrency)&oldid=1347382552
[25] “Tether (cryptocurrency),” *Wikipedia*. Mar. 27, 2026. Accessed: Apr. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Tether_(cryptocurrency)&oldid=1345722192
[26] “ERC-20 Token Standard,” ethereum.org. Accessed: Apr. 14, 2026. [Online]. Available: https://ethereum.org/developers/docs/standards/tokens/erc-20/
[27] “Receive payouts.” Accessed: Apr. 20, 2026. [Online]. Available: https://docs.stripe.com/payouts?locale=en-GB
[28] E. C. Bank, “Single Euro Payments Area (SEPA),” Dec. 2025, Accessed: Apr. 20, 2026. [Online]. Available: https://www.ecb.europa.eu/paym/retail/sepa/html/index.en.html

12. Appendix

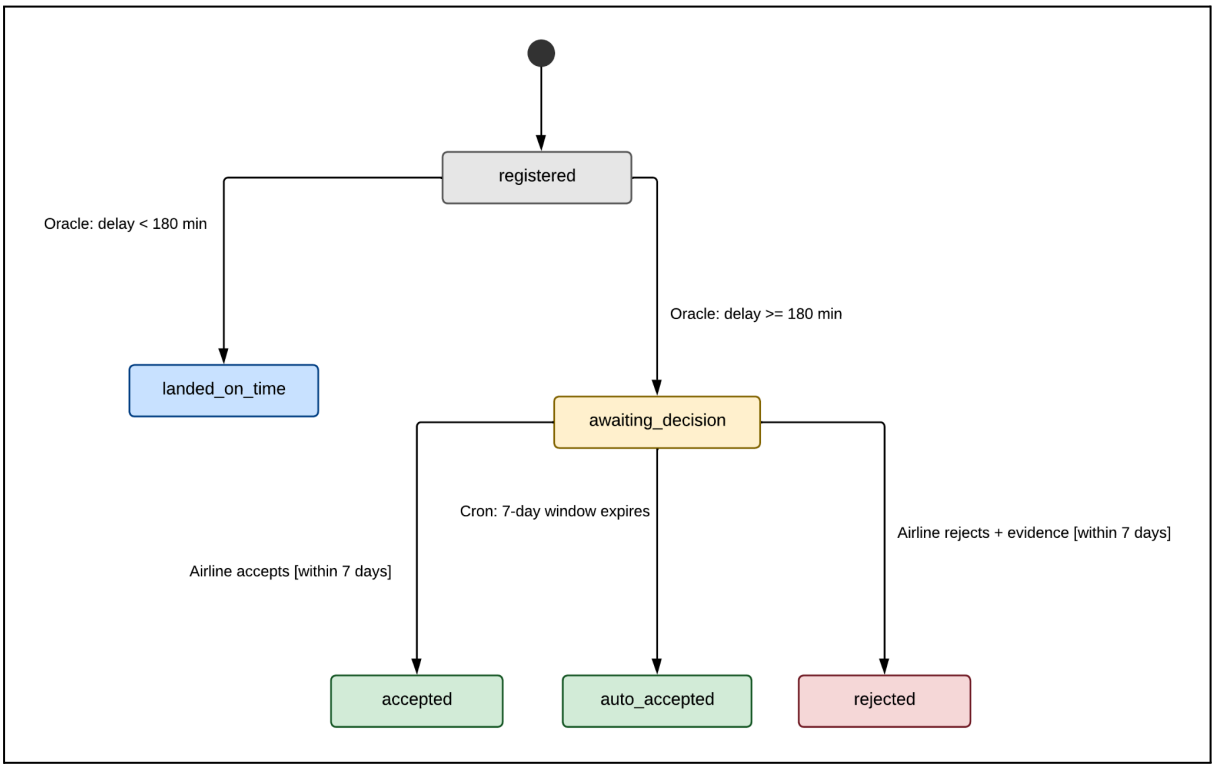

Figure 7: Claim lifecycle state machine showing all six states and the actors responsible for each transition.

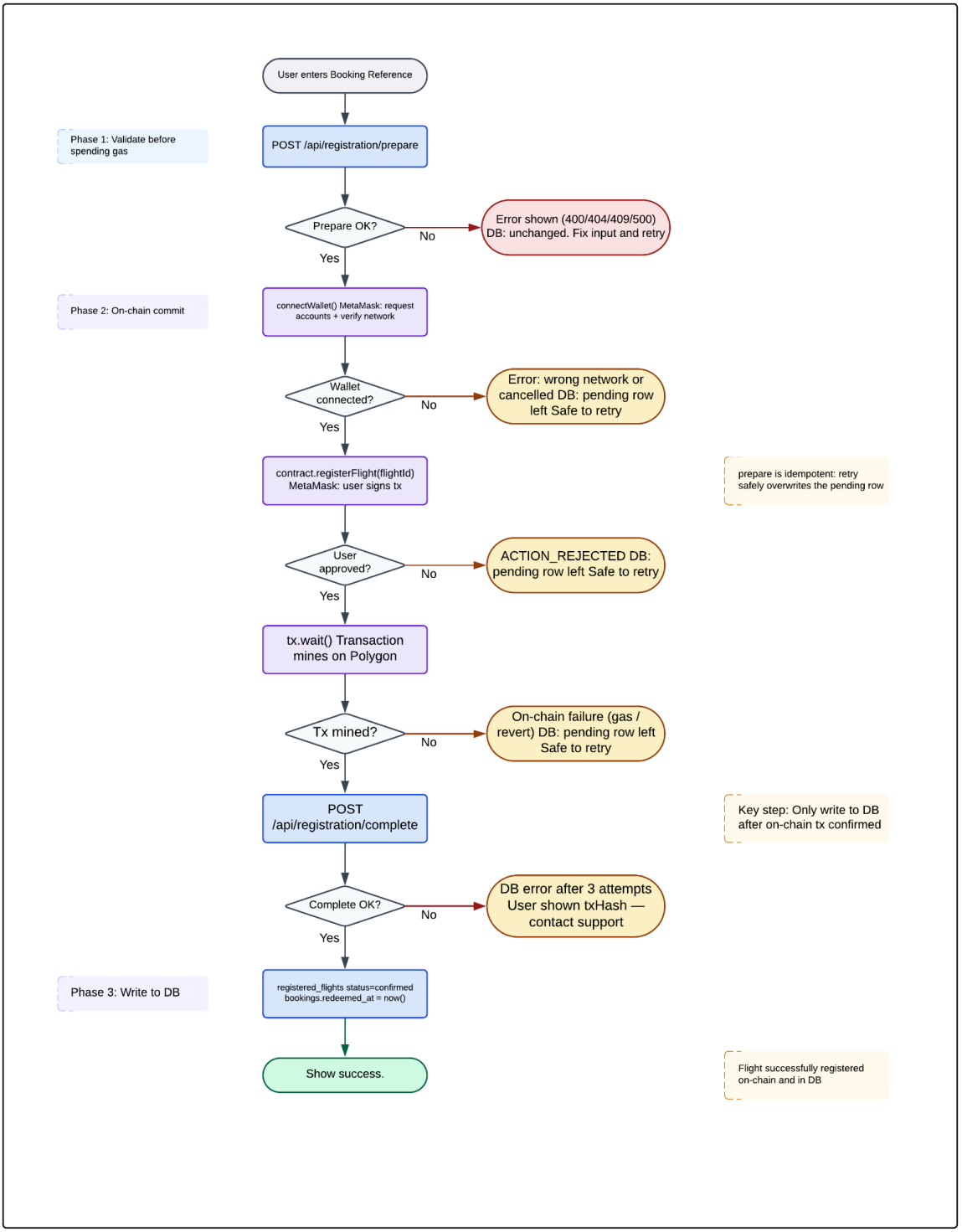

Figure 8: Two-phase registration flow showing the success path, failure branches, and DB state at each exit point.

User Testing Survey Questions:

Task 1 – Passenger

1. How easy was it to register your flight? *(1 = Very difficult, 5 = Very easy)*
2. How easy was it to create an account and login? *(1 = Very difficult, 5 = Very easy)*

Task 2 - Admin

3. How easy was it to record flight landing details and mark flights as delayed? *(1 = Very difficult, 5 = Very easy)*
4. How clear was the admin interface in terms of what you could do and what each action would affect? *(1 = Very unclear, 5 = Very clear)*

Task 3 – Airline

5. How easy was it to find flights that needed a decision? *(1 = Very difficult, 5 = Very easy)*
6. How clear was the process of accepting or rejecting a claim, including providing evidence for rejections? *(1 = Very unclear, 5 = Very clear)*
7. How useful did you find the airline profile / analytics view? *(1 = Not useful, 5 = Very useful)*

Task 4 – Passenger

8. How easy was it to access your evidence report? *(1 = Very difficult, 5 = Very easy)*
9. How useful did you find the passenger profile page? *(1 = Not useful, 5 = Very useful)*

General

10. How clear was the visual design and overall layout? *(1 = Very unclear, 5 = Very clear)*
11. How confident did you feel that the app kept you informed about what was happening at each step? *(1 = Not confident at all, 5 = Very confident)*

Open-Ended

12. Is there anything in the interface that confused you or that you felt was missing?
13. Any other suggestions for improvement?